Bare-metal MKR1000: a self-guided course

Christoph Ungricht

Contents

1 Bare-metal MKR1000: a self-guided course

A learning course for writing your own linker script, startup code, drivers, and eventually a WiFi server on the Arduino MKR1000 -- **without any SDK**. The philosophy is Socratic: hints instead of answers. You find addresses and bit positions in the datasheets, you write the code, you understand every line.

1.1 Target facts you'll keep referring to

- **MCU**: ATSAMD21G18A (inside the ATSAMW25 module), **Cortex-M0+** core.
- **WiFi controller**: ATWINC1500 (also inside the ATSAMW25 module), connected to the SAMD21 over SPI. This is a *separate chip*, not part of the SAMD21.
- **Three authoritative documents** you will live in:

1. **SAMD21 Family Datasheet** -- get it from Microchip ("SAM D21 Family Data Sheet", document DS40001882). Register-level reference for clocks, GPIO, NVIC, memory map.
2. **Cortex-M0+ Devices Generic User Guide** (`docs/Cortex-M0/dui0662a_...pdf`) -- core registers, vector table layout, exception model.
3. **Cortex-M0+ TRM** (`docs/Cortex-M0/DDI0484C_...pdf`) -- deeper core internals; secondary.

- **Board specifics** (LED pin, crystal, etc.): `docs/ABX00004/ABX00004-schematics.pdf` – `ABX00004-full-pinout.pdf`.
- **WiFi chip specifics**: `docs/ATSAMW25-MR210PB/Atmel-42618-...Datasheet.pdf` and the SAM W25 software programming guide in the same folder.

Action 0: open the SAMD21 datasheet's "Memory Organization" chapter and the "Physical Memory Map" figure. Write down on paper: flash base address, flash size, SRAM base address, SRAM size, peripheral region base. You will need these constants in every later module.

1.2 Course map

The course is organised in five parts plus a board-orientation prelude. Modules build on each other; do them in order unless a module explicitly says it's optional.

Part	Module	Topic
0. The board	0	The MKR1000 board itself: what's on it, what it's for
I. Context (read once, refer back)	1	A tour of bare-metal processor families
	2	Toolchains: open source and proprietary
	3	Tools for this course and what the Arduino script does
II. The boot path	4	Memories of the SAMD21
	5	Vector table and Cortex-M0+ boot sequence
	6	Writing your own linker script
	7	Startup in ARM assembly (your crt0)
	8	specs files and libc
III. First hardware	9	Blink: PORT/GPIO
	10	SysTick: your first proper peripheral
	11	Real clocks (48 MHz) and a UART
IV. Going further (needs SWD)	12	Debug probes: JTAG, SWD, and using one
	13	Writing your own bootloader
V. The big project	14	A HAL design and the road to a WiFi server
Appendices	A	Glossary
	B	GCC flags reference
	C	UART, I2C, SPI: bus protocols with practical examples

Part	Module	Topic
	D	Designing your own PCB (open-source tools + git)

1.3 Module 0 -- The MKR1000 board itself

Before diving into ARM cores and linker scripts, get a picture of the actual physical thing in front of you. The MKR1000 is **a specific product** with specific design goals; understanding them prevents wasting effort on the wrong abstraction later (e.g. trying to do audio DSP on it, or expecting onboard debug hardware).

1.3.1 0.1 What the board is, and what it's for

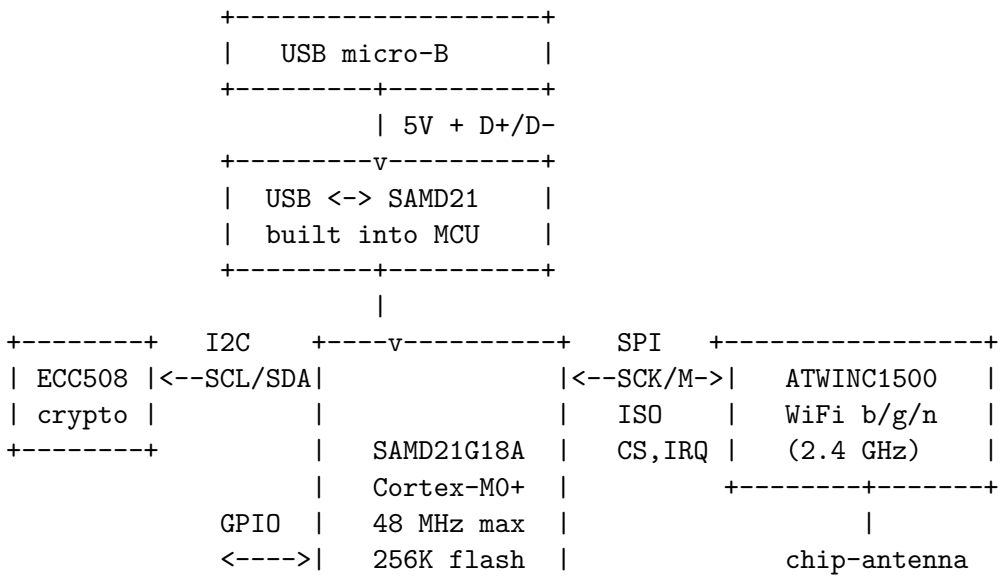
The MKR1000 (Arduino product ID **ABX00004**) is one of the early boards in Arduino's **MKR family**: a 67.6 mm x 25 mm form factor designed for **battery-powered, WiFi-connected IoT prototypes**. The defining decisions of the design:

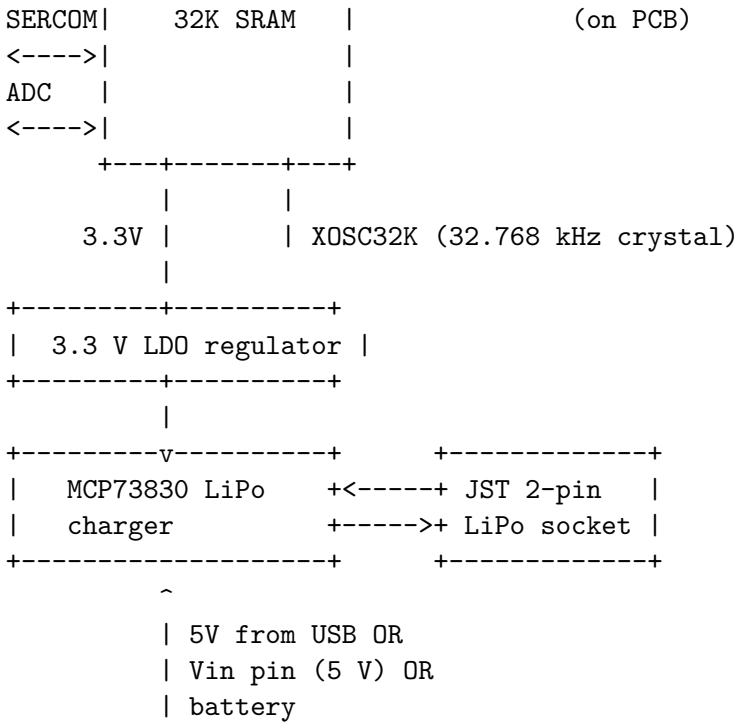
- **A modern 32-bit ARM MCU** (SAMD21G18A) instead of the AVR found on Uno.
- **WiFi built in**, not as a shield.
- **A LiPo battery socket and onboard charger**, so the same board can run from USB or from a single-cell battery, with charging seamless when USB is plugged in.
- **3.3 V I/O only** (not 5 V-tolerant on most pins -- check the pinout before connecting 5 V sensors).
- **A crypto chip** so devices can hold a unique cryptographic identity for TLS / authenticated cloud connections, without bit-banging keys yourself.

It is positioned for: small WiFi nodes (sensors, switches, displays), learning embedded networking, classroom IoT projects, prototypes that may move to a custom PCB later. It is **not** positioned for: heavy compute, video, audio DSP, hard real-time motor control faster than a few kHz, sub-100 uA sleep budgets (the WINC1500 by itself draws non-trivial standby current).

1.3.2 0.2 Block diagram

Read this top-down. Each block is documented in detail in `docs/`.





The three on-board peripherals you'll talk to from your SAMD21 firmware:

- **ATWINC1500** -- the WiFi controller, on SERCOM-SPI. Module 14 of this course is about getting it to work.
- **ATECC508A** -- the secure-element crypto chip, on SERCOM-I2C. Holds per-device ECC private keys that never leave the chip; can sign, verify, do ECDH. Designed for cloud-IoT authentication. *Not* covered in detail here; once you have I2C working (Appendix C), the chip's datasheet is the reference.
- **MCP73830** -- the LiPo charge controller. *You do not talk to it.* It manages the battery autonomously. Its only "interface" is a status pin the SAMD21 can read.

1.3.3 0.3 The components, one paragraph each

1.3.3.1 SAMD21G18A (the MCU) ATMEL/Microchip 32-bit Cortex-M0+ at up to 48 MHz. 256 KiB flash, 32 KiB SRAM. Rich peripheral set including six **SERCOM** modules (each configurable as UART, SPI, or I2C), three timer/counters, a DAC, ADC, USB device, RTC. Lives inside the **ATSAMW25** module on the board (the metal-can shield), together with the WINC1500. Documents: search for "SAM D21 Family Data Sheet" (Microchip DS40001882).

1.3.3.2 ATWINC1500 (the WiFi controller) Microchip's IEEE 802.11 b/g/n 2.4 GHz radio + MAC + on-chip TCP/IP stack + TLS, with its own ARM CPU running its own firmware. The SAMD21 talks to it over SPI; the WINC1500 does almost all the WiFi heavy lifting itself. Antenna is a small chip antenna on the PCB. See `docs/ATSAMW25-MR210PB/Atmel-42618-...Datasheet.pdf` and the software programming guide in the same folder.

1.3.3.3 ATECC508A (the crypto chip) Microchip's secure element: stores up to 16 ECC P-256 private keys in tamper-resistant storage, performs ECDSA sign/verify and ECDH, has a hardware random-number generator. Talks I2C. The point: a device can hold a unique, unclonable cryptographic identity that even your own firmware cannot read out. Used in cloud-IoT TLS mutual authentication. Datasheet: Microchip's website (not in your repo; download separately).

1.3.3.4 MCP73830 (the LiPo charge controller)

Single-cell Li-Ion / Li-Po linear charger. Charge current set by an external resistor on its PROG pin. Handles USB-powered charging autonomously: connect USB while a LiPo is plugged in and it charges the battery while powering the rest of the board. Datasheet in `docs/MCP73830/`.

1.3.3.5 32.768 kHz crystal (XOSC32K)

A tiny watch-crystal on the PCB. Used by the SAMD21's DFLL48M as a stable, low-jitter reference, so you can generate an accurate 48 MHz CPU clock and an accurate millisecond/RTC timebase. Without it you'd be stuck on the internal 8 MHz oscillator. You will configure this in Module 11.

1.3.4 0.4 Power architecture

Three possible power sources, in priority order:

1. **USB micro-B (5 V)** -- normal "plug into laptop" case.
2. **Vin pin (5 V)** -- if you're embedding the MKR1000 in a larger board that supplies 5 V.
3. **LiPo battery on the JST socket (3.0 - 4.2 V)** -- single-cell only.

A 3.3 V LDO regulator generates the rail that powers everything on the board. The MCP73830 sits between USB/Vin and the LiPo socket, so:

- USB + no battery: regulator runs from USB-5V, MCP73830 idle.
- USB + battery: regulator runs from USB-5V, MCP73830 charges the battery from USB-5V, board operates normally.
- No USB + battery: regulator runs from the battery (3.0--4.2 V), no charging.

The Vcc rail your firmware sees is always **3.3 V** regardless of source. The board's **VCC pin** brings out 3.3 V (for powering external sensors); **5V** is brought out too but only available when USB or Vin is feeding the board.

> Action: open `docs/ABX00004/ABX00004-schematics.pdf` and trace the path from the USB connector through to the SAMD21's VDD pin. Find the LDO part number and its decoupling caps. This is genuinely useful muscle memory before you design your own carrier (Appendix D).

1.3.5 0.5 I/O at a glance

Numbers below are illustrative; **confirm against `docs/ABX00004/ABX00004-full-pinout.pdf`** before wiring anything up.

- ~22 digital I/O pins (mix of pure GPIO and shared with peripherals).
- 7 analog input pins routed to the SAMD21's ADC.
- 1 true DAC output (the SAMD21 has one 10-bit DAC).
- PWM available on many pins via TC/TCC peripherals.
- USB device (the same port you flash through can also be a CDC serial device, HID device, etc.).
- One UART, one SPI, one I2C *brought out to the headers* -- each is one SERCOM configured for that role. Other SERCOMs are used internally for WiFi (SPI) and crypto (I2C).
- SWCLK / SWDIO debug pads (Module 12 will use these).
- All I/O is **3.3 V**. Many pins are **not 5 V tolerant** -- check before connecting 5 V sensors. Use a level shifter or a resistive divider if needed.

1.3.6 0.6 What you can realistically build with this

Practical project ideas matched to the board's strengths:

- **WiFi sensor node**: read a sensor every minute, POST to a server, sleep. Battery life of weeks is achievable with careful sleep modes.
- **Networked switch / relay**: serve a small HTTP page, control a GPIO from a browser.
- **MQTT publisher** to a home-automation system (Home Assistant, Node-RED).
- **TLS-authenticated cloud client** using the ATECC508A for the private key.
- **USB HID device** with WiFi-configurable behaviour.
- **Test bench remote**: tiny Web UI for triggering instruments.

What it is **not** the right tool for:

- Anything that wants > 256 KiB code or > 32 KiB working data.
- Audio DSP beyond toy bit-banged sound (no dedicated audio peripheral; M0+ has no FPU and only software floats).
- Video, graphics, large displays (no LCD controller, no MIPI).
- Hard real-time control loops above a few kHz that *also* need WiFi -- the WINC1500's SPI traffic will jitter you.
- Battery-powered designs needing single-digit-microamp sleep -- the WINC1500 is the limit.

  Action: pick one realistic project for *yourself* to aim at while you go through this course. The course will get you to "blink" and "WiFi server" along the way, but having a concrete personal target makes the abstract parts (linker scripts, startup, HAL) feel motivated.

1.4 Module 1 -- A tour of bare-metal processor families

Context first: where does the Cortex-M0+ on your MKR1000 sit in the landscape of chips you might program bare metal? You'll meet most of these eventually.

1.4.1 The ARM Cortex families

ARM Ltd. designs CPU cores; companies like Microchip (your SAMD21), ST (STM32), NXP, Nordic (nRF52/nRF53), Raspberry Pi (RP2040/RP2350), Apple, and Qualcomm license them and add their own peripherals. ARM splits the Cortex line into three classes:

Class	Examples	Architecture profile	What it's for	Has MMU?	Runs Linux?
Cortex-M (microcontroller)	M0, **M0+** (your chip), M1, M3, M4, M7, M23, M33, M55, M85	ARMv6-M, ARMv7-M, ARMv8-M	MCUs: deterministic, low power, runs from flash	No (MPU optional)	No
Cortex-R (real-time)	R4, R5, R7, R8, R52, R82	ARMv7-R, ARMv8-R	Hard real-time: automotive ECUs, cellular baseband, storage controllers	MPU; some have MMU	Rarely (specialized RTOSes)

Class	Examples	Architecture profile	What it's for	Has MMU?	Runs Linux?
Cortex-A (applications)	A7, A53, A55, A72, A76, A78, A510, A720, A920	ARMv7-A, ARMv8-A, ARMv9-A	Application processors: phones, Raspberry Pi, set-top boxes	Yes (full MMU + caches)	Yes (Linux/Android)

Inside Cortex-M, the **instruction-set** subdivision matters more than the core number:

- **ARMv6-M** (M0, M0+, M1, M23): Thumb-1 + a small handful of Thumb-2. Very limited addressing modes. No integer divide instruction. Only 16-bit instructions plus a few 32-bit ones. This is your MKR1000.
- **ARMv7-M** (M3, M4, M7): full Thumb-2, hardware divide, bit-banding (M3/M4), optional FPU (M4F, M7F), optional DSP instructions.
- **ARMv8-M** (M23, M33, M55, M85): adds TrustZone-M (secure/non-secure worlds), more instructions, optional Helium (MVE) vector extension on M55/M85.

  Practical consequence for you: code you write today for M0+ will run on any larger Cortex-M, but not vice versa. A `udiv` instruction in M4 code will fault on M0+.

1.4.2 Other architectures you'll encounter in bare-metal work

Family	Vendor	Bit width	ISA style	Typical board you'd see
AVR	Atmel (now Microchip)	8-bit	Harvard, custom AVR ISA, ~130 instructions	Arduino Uno (ATmega328P), Arduino Mega
PIC8/16/32	Microchip	8/16/32-bit	Harvard, multiple families with incompatible ISAs	PICkit boards, lots of industrial
MSP430	Texas Instruments	16-bit	von Neumann, very low power, RISC-ish	TI LaunchPad
8051 / MCS-51	Originally Intel, now many vendors (Silicon Labs, NXP)	8-bit	Harvard, 1980s ISA still in use	EFM8 LaunchPad
RISC-V	Open ISA, many vendors (SiFive, Espressif, Bouffalo, GD32)	32 or 64-bit	Clean modern RISC, modular extensions	ESP32-C3/C6, BL602, HiFive boards
Xtensa LX6/LX7	Tensilica (Cadence)	32-bit	Configurable RISC	ESP8266, ESP32 (original), ESP32-S2/S3
AVR32 / SuperH / Blackfin / MIPS	various	32-bit	mostly legacy now	older embedded, set-top boxes

Family	Vendor	Bit width	ISA style	Typical board you'd see
x86 / x86-64	Intel, AMD	32/64-bit	CISC	PC-class bare metal (BIOS, hobby OS dev)
PowerPC (e200, e500)	NXP, IBM	32/64-bit	RISC	automotive (MPC57xx), older game consoles

1.4.3 Architectural axes that matter when you switch chips

When you move bare-metal work to a different chip, these are the questions you have to re-answer:

1. **Harvard vs von Neumann.** Harvard architectures (AVR, classic PIC) have separate address spaces for code and data; reading a constant from "flash" is not just a normal pointer dereference (`pgm_read_byte` on AVR). Cortex-M is von Neumann (unified address space) -- `const char*` to flash just works.
2. **Word size and pointer size.** 8-bit AVRs have 16-bit pointers; 32-bit Cortex-M has 32-bit pointers; 64-bit Cortex-A has 64-bit pointers. Affects struct layout, ABI, and how much you can sanely point at.
3. **Endianness.** ARM and RISC-V are configurable but almost always **little- endian** in practice. Some PowerPC and older MIPS were big-endian. Matters for memory dumps and binary protocols.
4. **Interrupt model.** Cortex-M's NVIC vectors directly to C functions and handles stack-frame save/restore in hardware. AVR and classic ARM (pre-Cortex) require you to write assembly prologues that save registers yourself.
5. **Memory protection.** None on M0+; optional MPU on M3+; full MMU on Cortex-A. Affects whether "bare metal" can mean "with virtual memory."
6. **Privilege levels.** M0+ has two (privileged/unprivileged) but most bare- metal code stays privileged. Cortex-A has EL0-EL3 and the secure world, which is a whole subject on its own.
7. **Toolchain triple.** `arm-none-eabi-gcc` for ARM bare metal (your toolchain), `avr-gcc` for AVR, `riscv64-unknown-elf-gcc` for RISC-V, `xtensa-esp32-elf-gcc` for ESP32 Xtensa. Different triples, mostly the same commands.

1.4.4 What the different assembly languages look like

A blink-style "toggle a bit" sequence in different ISAs to give you the flavour:

ARM Thumb (your MKR1000, ARMv6-M):

```
ldr   r0, =0x41004400   @ address of some register
ldr   r1, [r0]
movs  r2, #1
eors  r1, r2            @ XOR with 1
str   r1, [r0]
```

Fixed-length 16-bit instructions (mostly), only registers r0-r7 freely usable in most instructions, two-operand format (destination = destination OP source).

ARM A64 (Cortex-A, 64-bit):

```
ldr   x0, =0xFFFF000041004400
ldr   w1, [x0]
```

```
eor   w1, w1, #1
str   w1, [x0]
```

Fixed 32-bit instructions, 31 general-purpose registers (x0-x30, w0-w30 for 32-bit views), three-operand format.

AVR (ATmega):

```
in    r16, 0x05         ; read PORTB
ldi   r17, 0x20
eor   r16, r17
out   0x05, r16
```

8-bit registers (r0-r31), special I/O instruction space, fixed 16-bit instructions.

RISC-V (RV32I):

```
li    t0, 0x10012000
lw    t1, 0(t0)
xori  t1, t1, 1
sw    t1, 0(t0)
```

Fixed 32-bit instructions (compressed 16-bit optional via the `C` extension), 32 general-purpose registers, three-operand format, very regular encoding.

x86-64:

```
mov   rax, 0xFFFF800000001000
xor   dword ptr [rax], 1
```

Variable-length instructions (1-15 bytes), CISC with memory-operand ops, two-operand format.

> Why this matters: a "startup file in assembly" looks different per ISA. The *concept* (set stack pointer, copy `.data`, zero `.bss`, call main) is the same everywhere, but the instructions are not. The skill you build in Module 7 transfers directly to any Cortex-M and conceptually to all the others.

1.4.5 Why this chip, for learning

The Cortex-M0+ is a sweet spot for learning bare metal:

- Simple enough that the user guide fits in one PDF and you can hold the whole machine in your head.
- Modern enough (ARM, vector table, NVIC, SWD) that the patterns you learn transfer to STM32, nRF, RP2040, and the larger Cortex-M parts.
- Cheap, widely available, well-documented.
- Has a real bootloader and a real WiFi controller on the same module -- enough material to keep you busy for months.

1.5 Module 2 -- Toolchains: open source and proprietary

A **toolchain** is the bundle of programs you need to turn source code into a binary the chip can run: compiler, assembler, linker, librarian, objcopy/size utilities, debugger. For bare-metal ARM, the parts always include some flavour of:

- a **C compiler** (and optionally C++),

- an **assembler**,
- a **linker** (with linker-script support),
- a **C library** for freestanding/embedded use,
- a **debugger** that speaks to a debug server like OpenOCD.

There are roughly three camps: open-source GCC-based, open-source LLVM-based, and commercial. You can absolutely complete this entire course with only open-source tools -- that's what you're doing.

1.5.1 Open-source toolchains

1.5.1.1 GCC: arm-none-eabi-gcc (what you're using)

- Maintained by ARM as **GNU Toolchain for the Arm Architecture** (formerly "GNU Arm Embedded Toolchain", formerly Linaro builds). Downloadable from ARM's developer site. You already have version 15.2 at `/opt/arm-gnu-toolchain-15.2.rel1-x86_64-arm-none-eabi`.
- Triple `arm-none-eabi`: target is ARM, no OS ("none"), uses the Embedded ABI. The `eabi` matters: it dictates calling convention, struct layout, and how the compiler talks to its libc.
- Bundles: `arm-none-eabi-gcc` (C), `g++` (C++), `as` (assembler), `ld` (linker), `objcopy`, `objdump`, `size`, `nm`, `ar`, `ranlib`, `arm-none-eabi-gdb` (debugger), and **newlib** + **newlib-nano** as the bundled libc.
- Strengths: ubiquitous, free, supports every ARM core that exists, deeply integrated with `binutils`, very stable.
- Weaknesses: error messages can be terse; optimisation on Cortex-M0+ is decent but not best-in-class.

1.5.1.2 LLVM/Clang: clang + lld

- LLVM has had reasonable ARM bare-metal support for years. You'd typically invoke it as `clang --target=arm-none-eabi -mcpu=cortex-m0plus`.
- Toolchain pieces: `clang` (C/C++), `llvm-as`, `lld` (linker), `llvm-objcopy`, `llvm-objdump`, `lldb` (debugger).
- ARM also ships an LLVM-based commercial-ish distro called **LLVM-embedded Toolchain for Arm** (open source, MIT-licensed builds) -- a curated LLVM + picolibc bundle. This is the modern alternative to arm-none-eabi-gcc and is gaining adoption.
- libc choice: **picolibc** (a fork/successor of newlib designed for embedded; smaller, MIT-licensed). picolibc also works with GCC.
- Strengths: better diagnostics, faster compiles, often smaller code on modern cores, single binary targets many architectures (one `clang` does ARM, RISC-V, Xtensa-with-plugin, x86, ...).
- Weaknesses: linker-script compatibility with GNU ld is good but not 100%; some GCC-specific extensions in vendor headers need tweaks.

1.5.1.3 Smaller / niche open-source toolchains

- **sdcc** (Small Device C Compiler): for 8051, Z80, STM8, classic PIC. Not ARM. Mention only because if you ever pick up an 8051 dev kit, this is what you'll use.
- **avr-gcc**: AVR fork of GCC. Bundled in Arduino IDE installations for AVR-based boards (Uno, Mega).

1.5.2 Proprietary / commercial toolchains

These are common in regulated industries (automotive, medical, aerospace) where certification, professional support, and tight IDE integration matter more than license cost.

Toolchain	Vendor	Compiler core	Typical use
Arm Compiler 6 (armclang)	Arm Ltd.	LLVM-based, proprietary	Used in **Keil MDK** IDE. Generates very tight code. Was the dominant commercial ARM compiler.
IAR Embedded Workbench (iccarm)	IAR Systems	proprietary	Popular for safety-critical work. Excellent debugger, certified versions for ISO 26262 / IEC 61508.
Segger Embedded Studio + emCompiler	Segger	based on Clang	Free for non-commercial use. Tightly integrated with J-Link.
MPLAB XC32	Microchip	based on GCC (with proprietary additions and an optional licensed optimiser)	Microchip's official compiler for SAMD/SAME/PIC32. The unlicensed version is essentially `arm-none-eabi-gcc` for SAM parts.
TI Code Composer Studio (cl2000, cl430, ticlang)	Texas Instruments	proprietary + Clang-based	For TI parts (MSP430, C2000, Sitara, CC13xx).
Renesas CC-RX / CC-RL	Renesas	proprietary	For Renesas RX, RL78.
ImageCraft, Tasking, Cosmic, HighTec	various	proprietary	Various niches (8051, ARM, PowerPC automotive).

1.5.3 IDEs (separate from the toolchain)

People often conflate "toolchain" and "IDE", but they're different:

- **VS Code + cortex-debug extension** with arm-none-eabi-gcc + OpenOCD -- open source, the modern hobby/professional default outside the regulated world.
  - *Pro*: free, cross-platform, huge ecosystem, you keep your text editor.
  - *Con*: configuration is per-project JSON; less hand-holding than vendor IDEs.
  - *Pick when*: you want a portable, scriptable, vendor-neutral setup.
- **Neovim + clangd + DAP plugin** -- same idea as VS Code, terminal-native.
  - *Pro*: keyboard-driven, lightest weight, works over SSH.
  - *Con*: assembly required to wire DAP + OpenOCD + clangd; lone-wolf territory.
  - *Pick when*: you already live in Vim/Neovim.
- **Keil MDK** (Windows) -- uses Arm Compiler 6 by default, can be switched to gcc.
  - *Pro*: best-in-class Cortex-M debugger UI; production-grade.
  - *Con*: Windows-only; size-limited free Community edition; commercial licence for unrestricted use; closed source.
  - *Pick when*: industrial work, regulated environments, vendor demands it.
- **STM32CubeIDE** -- Eclipse-based, ships with arm-none-eabi-gcc, free, ST specific.

  - *Pro*: tight integration with STM32 BSPs; CubeMX peripheral configurator is genuinely useful; free.
  - *Con*: Eclipse heaviness; ST-specific.
  - *Pick when*: you're working on STM32 parts.
- **MPLAB X IDE** -- NetBeans-based, ships with XC compilers, free, Microchip specific.
  - *Pro*: vendor-supported for SAMD21 (your part); MPLAB Harmony framework integrates with it.
  - *Con*: NetBeans heaviness; the licensed XC32 optimiser costs money.
  - *Pick when*: you want Microchip's official Harmony stack or debugging-tool integration (PICkit/ICD).
- **PlatformIO** -- package manager + build front-end over arm-none-eabi-gcc/ others. Integrates with VS Code.
  - *Pro*: one `platformio.ini` describes the target; cross-board work is very easy; works for SAMD21 too.
  - *Con*: another abstraction layer between you and your toolchain; harder to do non-standard things.
  - *Pick when*: you're prototyping across multiple board types and don't want to maintain N Makefiles.

1.5.4 libc options on bare-metal ARM

This is a frequent point of confusion, so it's worth separating from compiler choice:

- **newlib**: full POSIX-ish C library, large.
  - *Pro*: complete; behaves like a hosted C library; widely tested.
  - *Con*: heavy on flash, drags in a `malloc` that may be more than you want, and its `printf` is enormous if you enable floats.
  - *Use when*: you don't care about flash size and want maximum compatibility with desktop-style C code.
- **newlib-nano**: stripped-down newlib (smaller `printf`, no `wchar_t` by default, simpler `malloc`). Selected by `--specs=nano.specs`.
  - *Pro*: dramatically smaller than newlib (tens of KiB savings on `printf` alone); still fully featured enough for most embedded code.
  - *Con*: `printf` float support is opt-in via a separate spec; some locale features missing.
  - *Use when*: default for this course and almost any Cortex-M project that wants stdio without bloat.
- **picolibc**: a newer, smaller, MIT-licensed embedded libc. Default in the LLVM-embedded toolchain. Works with GCC too.
  - *Pro*: even smaller than newlib-nano; cleaner internals; MIT licence (newlib is BSD/GPL mix).
  - *Con*: smaller community than newlib; some niche functions missing.
  - *Use when*: you want the smallest possible binary or you're using the LLVM-embedded toolchain.
- **musl, glibc**: full-system libcs. **Not for bare metal.**
  - *Use when*: you've moved to Linux user space (the Linux course).
- **No libc** (`-nostdlib -nostartfiles -nodefaultlibs`).
  - *Pro*: zero external dependency; smallest possible image; you understand every byte.
  - *Con*: you write your own `memcpy`, `memset`, and any string handling you need. `printf` is your problem.
  - *Use when*: Module 8 of this course; first-time-bare-metal learning; or in firmware where every KiB matters.

1.5.5 Recommendation for this course

- **Compiler**: `arm-none-eabi-gcc 15.2` (what you have). Don't switch mid-course.
- **Linker**: GNU `ld` (comes with above).
- **Debugger**: `arm-none-eabi-gdb` (comes with above).
- **Debug server (when you get a probe)**: OpenOCD.
- **libc**: start with none (Module 8), add newlib-nano when you need `printf` (Module 11).
- **Editor**: Neovim with `clangd` (clangd targets the same files GCC compiles, so you don't have to switch compilers to get LSP).

Later, as an exercise, try rebuilding the same project with `clang --target=arm-none-eabi` + `lld` + picolibc. Compare binary sizes and disassembly. It's a great way to demystify "the toolchain" -- once you've swapped one, the abstraction is no longer scary.

1.6 Module 3 -- Toolchain, flashing tool, and what the Arduino script does

1.6.1 3.1 Read `arduino-flash.sh` like a recipe

Each line is one stage of a normal embedded build:

- **Lines 3 and 4**: compile each `.cpp` to `.o`. Note the flags `-mcpu=cortex-m0plus -mthumb -ffunction-sections -fdata-sections -nostdlib --specs=...` -- you will reuse most of them.
- **Line 5**: link with a **linker script** (`flash_with_bootloader.ld`) and **specs files** (`nano.specs`, `nosys.specs`) producing an `.elf`. `-Wl,--gc-sections` discards unused sections (paired with the per-function/data sections above).
- **Line 6**: `objcopy -O binary` -> raw `.bin` (what the bootloader wants).
- **Line 7**: `objcopy -O ihex` -> Intel HEX (alternative format).
- **Line 8**: `size -A` prints section sizes.
- **Line 9**: `bossac ... -U true -i -e -w -v ... -R` -- erase, write, verify, reset.

  Action: open `flash_with_bootloader.ld` (find it under `~/.arduino15/.../variants/mkr1000/linker_scrip`
  Don't try to understand it yet. Just notice it exists and that the name implies "the bootloader lives at the bottom of flash, my app starts higher up."

1.6.2 3.2 Pick a flashing tool

You have two realistic options on Void Linux:

Tool	How it talks to the MCU	Needs extra hardware?
bossac (open source, shumatech/BOSSA)	Through the **SAM-BA bootloader** already in flash, via USB CDC (`/dev/ttyACM*`)	No
OpenOCD	Through **SWD** (Serial Wire Debug) pins	Yes -- a CMSIS-DAP / J-Link / ST-Link probe wired to the SWD pads

For learning bare-metal blink without buying hardware: **use bossac**. It's already what Arduino uses, it is open source, and Void packages it (verify the exact name with `xbps-query -Rs bossa`).

OpenOCD becomes valuable later when you want **single-step debugging with GDB**. At that point you'll need an SWD probe and you'll connect it to the SWD test points on the MKR1000 (find them in `ABX00004-schematics.pdf`).

> Action: install bossac, plug in the MKR1000, double-tap reset to enter the bootloader (the board enumerates a different USB device in bootloader mode -- observe with `dmesg -w`), and run `bossac --help`. Confirm you can see the chip with `bossac -i -p ttyACM0`.

1.6.3 3.3 The bootloader changes everything about your linker script

The MKR1000 ships with the SAM-BA bootloader in the first part of flash. Your application **cannot** start at the flash base -- it has to start where the bootloader hands off control.

> Action: find in `flash_with_bootloader.ld` the offset where the application's flash region begins. Write it down. (Hint: it'll be a small power-of-two offset from the flash base. Cross-check it against the bootloader size mentioned in the Atmel SAM-BA documentation or the comments in the linker script itself.)

1.6.4 3.4 Output formats: ELF, HEX, BIN -- when do you use which?

Lines 5-7 of `arduino-flash.sh` produce **three different files** from the same compilation: `.elf`, `.bin`, `.hex`. They contain the same machine code, packaged differently. Knowing which to use when is one of those little things that confuses everyone exactly once.

1.6.4.1 `.elf` -- the rich object file **Executable and Linkable Format.** This is the linker's native output.

- **Contains**: machine code, data sections, **plus** the full ELF metadata: section headers, symbol table, debug info (DWARF), section load addresses, relocation tables, build attributes (`.ARM.attributes`).
- **Size on disk**: largest of the three (debug info dominates -- in our blink demo, ~20 KiB on disk for ~300 bytes of actual code).
- **Loadable directly onto the chip? No** -- ELF is a *container* describing where code should go, not a flat image the chip can execute. You need `objcopy` to flatten it.
- **What it's used for**:
  - `arm-none-eabi-gdb` connects to the chip via OpenOCD and uses the `.elf` to know symbol names, function boundaries, source line numbers. **Without the `.elf`, GDB sees raw addresses.**
  - `arm-none-eabi-size`, `objdump`, `nm`, `readelf` all consume `.elf`.
  - The `.map` file pairs with the `.elf` for size auditing.
- **Keep the `.elf` forever.** It's how you'll debug a binary that shipped six months ago.

1.6.4.2 `.bin` -- the raw flash image **Flat binary.** Exactly the bytes that should end up in flash, starting at the application's load address.

- **Contains**: only the data that gets programmed -- `.isr_vector`, `.text`, `.rodata`, and the LMA of `.data`. No metadata, no symbols, no debug info, no address information at all.
- **Size**: smallest. For our blink: 280 bytes.
- **Loadable directly?** Yes -- but the flasher has to know *where* to put it, because the file has no addresses inside it. For bossac and the MKR1000 bootloader: flash from offset 0x2000 (the start of the application region after the bootloader).
- **Bossac speaks `.bin`** -- this is the format the SAM-BA bootloader expects. Line 9 of `arduino-flash.sh` feeds `.bin` to bossac.

- **Pitfall**: if there are gaps in your image (e.g. you put some data in a high flash region with a hole below), `.bin` either fills the gap with zeros (large file) or just can't represent it (you'd need HEX).

1.6.4.3 `.hex` -- the universal address-aware text format **Intel HEX format.** ASCII text. Each line is a "record":

`:LLAAAATT[DD...DD]CC`

- `LL` = byte count of data in this record (hex).
- `AAAA` = 16-bit address (extended addresses use record types 02/04 to set the upper bits).
- `TT` = record type: 00 data, 01 EOF, 04 extended linear address, etc.
- `DD..DD` = the data bytes.
- `CC` = checksum.
- **Contains**: code + load addresses, in a sparse format. Can describe "load these bytes at 0x2000, these other bytes at 0xFC00" with no zeros in between.
- **Size**: bigger than `.bin` (text overhead is ~2.2x), smaller than `.elf` (no debug info).
- **Loadable directly?** Yes, by any flasher that speaks HEX -- which is almost all of them.
- **Used by**: most JTAG/SWD programmers (J-Flash, OpenOCD's `program` command, `srecord/bincopy` tools), in-circuit programmers, third-party uploader tools that don't know your chip's flash base.

1.6.4.4 `.srec` -- the Motorola alternative You'll occasionally see **Motorola S-Record** (`.srec`, `.s19`, `.s28`, `.s37`). Same idea as Intel HEX -- text, address-aware -- different record format. More common on PowerPC/68K, less so on ARM. Convert with `objcopy -O srec`.

1.6.4.5 `.uf2` -- the drag-and-drop format **Universal Flash Format**, Microsoft's design. Used by RP2040, ESP32-S2/S3 in download mode, Adafruit's nRF and SAMD bootloaders. Lets a USB mass-storage bootloader appear as a drive; you literally drag the `.uf2` onto it and the bootloader handles the rest.

The MKR1000's stock bootloader does not speak UF2 -- it's SAM-BA, which is what bossac is for. Useful to know about because Pi Pico and many modern boards do use UF2, and the format is convertible from BIN with `uf2conv.py`.

1.6.4.6 Quick decision table

You want to ...	Use
Flash via `bossac` (MKR1000 default)	`.bin`
Flash via OpenOCD `program` command	`.elf` (preferred) or `.hex`
Flash via a generic JTAG/SWD programmer (J-Flash, ...)	`.hex`
Debug with GDB / inspect with `objdump/nm/size`	`.elf`
Compare two builds to see what changed	`.elf` + `size/nm/objdump`; `.bin` + `cmp/sha256sum` for byte-identity
Drag-and-drop onto a USB mass-storage bootloader	`.uf2` (not for stock MKR1000)

You want to ...	Use
Archive "what we shipped to fab"	`.elf` (debuggable) **and** `.bin` (byte-identical-to-flash)

1.6.4.7 A note on equivalence For the MKR1000 you can rebuild any of the three from the `.elf`. So in CI, **build the `.elf` once and `objcopy` it to whichever flavours you need**. The reverse is not true -- a `.bin` has lost the symbol table and you can't get the `.elf` back from it.

> Action: from your `blink.elf`, generate `.bin` and `.hex` with `objcopy`, then compare their sizes. Open the `.hex` in a text editor and find the first data record -- the address in the second field should be your application's load address (`0x2000` on this board).

1.6.5 3.5 Flashing each format onto the MKR1000

Now the *how*. Different tools accept different formats. Here are copy-pasteable commands for every realistic path, with the necessary preconditions.

1.6.5.1 Path A: bossac over USB (`.bin`) -- the no-extra-hardware path This is what `arduino-flash.sh` does and what you'll use day to day if you don't have an SWD probe.

Precondition: the SAM-BA bootloader must still be on the board (the factory state). The board must be *in bootloader mode* -- press the reset button **twice in quick succession** ("double-tap reset"), which causes the bootloader to stay resident and enumerate.

```
# 1. Double-tap reset on the MKR1000. Then verify it enumerated:
ls /dev/ttyACM*                 # should show your board
dmesg | tail                    # confirms the bootloader VID/PID

# 2. Sanity-check bossac sees it:
bossac -p ttyACM0 -i            # prints chip info, no flashing

# 3. Flash:
bossac -p ttyACM0 \
       --erase --write --verify --reset \
       -U true -i \
       blink.bin
```

Flag breakdown: - `--erase` (`-e`): erase application area first. - `--write` (`-w`): program the file. - `--verify` (`-v`): readback-verify. - `--reset` (`-R`): jump to the application after writing. - `-U true`: USB port (vs. native serial port mode). - `-i`: print info as it works.

You **cannot** feed `.elf` or `.hex` to bossac directly -- it expects flat binary at offset 0x2000 (the application-region start, which is the only address it programs). Use `.bin` here, full stop.

> Action: actually do this with the `blink.bin` you built in `/tmp/ mkr1000-blink/`. Watch the LED start blinking.

1.6.5.2 Path B: OpenOCD direct, one shot (`.elf` preferred, `.hex` also fine, `.bin` with an explicit address) **Precondition**: SWD probe wired up (Module 12). OpenOCD installed. This works **even if the bootloader is erased** -- it goes through SWD, not USB. You also don't need to put the board in bootloader mode.

```
# OpenOCD-only one-shot programming. Replace the config files with
# whatever your probe and target need (Module 12 walks through this).
OOCDCFG="-f interface/cmsis-dap.cfg -f target/at91samdXX.cfg"

# (a) From an .elf (the easiest path -- addresses come from the ELF):
openocd $OOCDCFG \
        -c "program blink.elf verify reset exit"

# (b) From a .hex (addresses come from the HEX records):
openocd $OOCDCFG \
        -c "program blink.hex verify reset exit"

# (c) From a .bin (you must supply the load address):
openocd $OOCDCFG \
        -c "program blink.bin 0x2000 verify reset exit"
```

The `program` command halts the target, erases the affected flash sectors, writes, verifies, then (with `reset`) reboots into your app.

> Pitfall: for the `.bin` form, **forgetting `0x2000`** results in a board where the bootloader and your app overlap or your app gets written to flash address 0 with no vector-table fix-up. The bootloader will then load nothing useful. If you erase the bootloader and write to 0x0000, you also need to update your linker script's FLASH `ORIGIN` to match.

1.6.5.3 Path C: GDB via OpenOCD (`.elf`) -- the debug workflow

Precondition: same as Path B (SWD probe + OpenOCD). This is the path you'll use during active development once you have a probe.

```
# Terminal 1: leave OpenOCD running.
openocd -f interface/cmsis-dap.cfg -f target/at91samdXX.cfg

# Terminal 2: start GDB on the .elf.
arm-none-eabi-gdb blink.elf
```

Inside GDB:

```
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt
(gdb) load              # programs flash from the .elf's sections
(gdb) monitor reset init
(gdb) break main
(gdb) continue
```

`load` reads the .elf, writes each loadable section to its specified address, and verifies. You're now sitting at a breakpoint on the first instruction of `main`. **This is the workflow once you have a probe.** Step through, inspect, fix, edit + rebuild + `load` again.

A `.gdbinit` in your project saves the typing:

```
target extended-remote localhost:3333
monitor reset halt
load
monitor reset init
```

Then `arm-none-eabi-gdb blink.elf` enters that state automatically.

1.6.5.4 Path D: Erase-then-flash from a totally bricked state

If you've written nonsense to flash (most commonly: erased the bootloader and now bossac can't see the board), here's the recovery from SWD:

```
# Halt the CPU first, then mass-erase, then program.
openocd $OOCDCFG \
        -c "init; reset halt; at91samd chip-erase; \
            program blink.elf verify reset exit"
```

The exact command for "chip erase" varies between OpenOCD target configs -- check `openocd ... -c "help"` against your target script. On the SAMD21 the relevant OpenOCD command is `at91samd chip-erase`.

If you want to reinstall the **bootloader** as well, flash both -- in this order:

```
openocd $OOCDCFG \
        -c "init; reset halt; \
            at91samd chip-erase; \
            program samd21_sam_ba_arduino_mkr1000.bin 0x0000 verify; \
            program blink.bin 0x2000 verify reset exit"
```

(The bootloader binary you'd get from `arduino/ArduinoCore-samd`'s `bootloaders/zero/` build output. Path/filename will differ.)

1.6.5.5 Summary table

Path	Format	Tool	Needs SWD probe?	Needs bootloader present?
A	`.bin`	`bossac` over USB	No	Yes
B-a	`.elf`	`openocd program`	Yes	No
B-b	`.hex`	`openocd program`	Yes	No
B-c	`.bin`	`openocd program <addr>`	Yes	No
C	`.elf`	`gdb load` via OpenOCD	Yes	No
D	`.elf`/`.bin`	`openocd` with `chip-erase`	Yes	No (recovery path)

> Action: until you have a probe, you live entirely on Path A. Pin this table somewhere visible the day you add a probe -- Paths B-C are the productivity unlock.

1.6.5.6 Which format should *you* reach for, in practice

Strip away the options and the rules are short:

- **You're going through this course right now, no probe yet** -> build `.bin`, flash with bossac (Path A). One file, one tool, no surprises.
- **You've added a probe and want a single "flash and run" command** -> use `.elf` with OpenOCD (Path B-a). The `.elf` carries its own addresses; you cannot fat-finger the load address.
- **You're actively writing/debugging code** -> `.elf` via GDB + OpenOCD (Path C). Edit, rebuild, `load`, step, repeat.

- **Your tool only takes one of `.hex` or `.bin`** (some vendor flashers, factory programmers, third-party JTAG GUIs) -> use what it takes. Prefer `.hex` when given the choice; the embedded load address is one less thing to get wrong.
- **You're handing a binary to someone else** (factory, customer, archive) -> ship the `.elf` *and* the `.bin`. The `.elf` is for debugging the version in the field; the `.bin` is byte-identical to what's actually programmed and verifies with a hash.
- **You're writing a CI pipeline** -> have it always produce `.elf` -> `.bin` and `.hex` via `objcopy`. Archive all three. Use `.bin`'s SHA256 as the build artifact identity.

The rule of thumb: **`.elf` is for humans and debuggers; `.bin` is for flash; `.hex` is for tools that need to know addresses without parsing ELF.**

1.7 Module 4 -- Memories on the SAMD21

Before writing a linker script, internalize what memories actually exist on this chip and what each one is for. The Cortex-M0+ architecture defines a flat 32-bit address space; the SAMD21 maps various physical memories into specific regions of it.

1.7.1 Cortex-M architecture memory map (the big picture)

ARM defines (recommends, really) a standard partitioning of the 4 GiB address space for Cortex-M parts. Roughly:

Region	Address range	Typical use
Code	`0x00000000 - 0x1FFFFFFF`	Flash (where execution starts)
SRAM	`0x20000000 - 0x3FFFFFFF`	On-chip RAM
Peripheral	`0x40000000 - 0x5FFFFFFF`	Memory-mapped peripheral registers
External RAM / device / private peripheral	higher regions	Mostly unused on M0+, or used by the core's own NVIC/SysTick (PPB at `0xE0000000`)

The SAMD21 places its physical memories within these regions. Confirm exact ranges in the datasheet's "Physical Memory Map".

1.7.2 The physical memories on a SAMD21G18A

1.7.2.1 1. Flash (program memory)

- Size: 256 KiB on the G18A. Located at the start of the Code region.
- Non-volatile, executable, readable like normal memory.
- Erase granularity: **rows** (a row = 4 pages on this part). Write granularity: **pages**. You can write 0-bits anywhere within a page, but to write 1-bits back you must erase the whole row first. This matters when you write your own flash driver.
- Where your code, your `.rodata`, and the *initial values* of your `.data` live. (Initial `.data` values are *copied* from flash to SRAM by startup code -- they don't execute from flash.)

  Action: find in the datasheet's NVM chapter the exact **page size** and **rows per block**. Also find the absolute flash base address and confirm total size.

1.7.2.2 2. SRAM (volatile working memory)

- Size: 32 KiB on the G18A. Located at the start of the SRAM region (`0x20000000`).
- Contents are undefined at power-up but **survive a CPU reset** (this is the property the double-tap-reset bootloader trick relies on, see Module 13).
- Where `.data`, `.bss`, the stack, and the heap live at runtime.

1.7.2.3 3. NVM User Row

- A small (16 bytes on the SAMD21) special area of non-volatile memory at a fixed address (look it up -- it's in the very low address space, *not* in the main flash array).
- Contains *user-configurable fuses*: bootloader size, watchdog defaults, brown-out detector settings, EEPROM emulation size, lock bits for flash regions, and more.
- Read like normal memory, written with the NVM controller (one row at a time -- same erase-then-write dance as main flash).
- The bootloader's existence depends on the bootloader-size field here being non-zero. **This is why erasing flash via SWD can leave the bootloader-size fuse intact: the user row is separate.**

  Action: find the **"NVM User Row Mapping"** table in the datasheet. Write down the bit positions of `BOOTPROT` (bootloader protection size) and `EEPROM` (EEPROM emulation size).

1.7.2.4 4. NVM Calibration / Factory Row

- Another special area written at the factory with per-die calibration values: ADC linearity, DFLL coarse/fine values, USB pad calibration, the chip's unique 128-bit serial number.
- Read-only from your code's perspective.
- You'll read from here in Module 11 when configuring the DFLL48M (the recommended way is to load Microchip's factory-calibrated value rather than calibrating it yourself).

1.7.2.5 5. Peripheral memory region

- Starts at `0x40000000`. Each peripheral (PORT, SERCOM0, TC3, USB, ...) has a base address and a struct of registers laid out at fixed offsets from it.
- These are not RAM. Writes have side effects (start a transfer, enable a clock). Reads can have side effects too (clearing a status flag on read).
- You **must** access these as `volatile` from C, otherwise the compiler may reorder, combine, or skip your accesses.

1.7.2.6 6. Private Peripheral Bus (PPB)

- At `0xE0000000`. Contains the core's own registers: SysTick, NVIC, SCB (System Control Block, which holds VTOR among others).
- Documented in the **Cortex-M0+ user guide**, not the SAMD21 datasheet -- because it's part of the core, not the chip.

1.7.2.7 7. ROM (bootrom)?

- Unlike some larger SAM/STM32 parts, the SAMD21 does **not** have a separate ROM-resident bootloader. The "SAM-BA bootloader" on the MKR1000 lives in the regular flash, in the bottom 8 KiB (typically) protected by the `BOOTPROT` fuse. Confirm in the datasheet that there's no factory ROM bootloader on this family.

1.7.3 How this maps to your linker script

You will define two `MEMORY` regions: **FLASH** and **RAM**. Their origin and length come from items 1 and 2 above -- but with FLASH's `ORIGIN` shifted up by the bootloader size (Module 3.3) for as long as you're flashing via bossac. The other memories don't need linker regions: NVM user/calibration are accessed by their fixed absolute addresses through pointer macros; peripherals likewise; PPB likewise.

1.7.4 A note on silicon errata

Every released chip has bugs the documentation doesn't fully reflect. Microchip publishes a separate **SAMD21 Family Errata** document listing known silicon issues per die revision -- some affect peripherals you'll use (USB, DFLL, NVMCTRL, EIC have all had errata over the part's life). For a learning project the impact is small; for any production work, read the errata **before** you trust a peripheral's documented behaviour.

> Action: search Microchip's site for "SAM D21 Family Silicon Errata" and download the current revision. Skim it for entries that mention features you plan to use. Note the die revision marking on your specific MKR1000 (in `SYSCTRL`'s DSU/DID register, or printed on the chip).

1.8 Module 5 -- The vector table and the Cortex-M0+ boot sequence

Before writing any code, answer these from the Cortex-M0+ Generic User Guide (`dui0662a`):

1. When the core comes out of reset, what are the **first two 32-bit words** it reads from memory, and what does it do with them?
2. Where is the vector table located by default after reset? (Hint: it's the same as the flash base on this part -- but think about how this interacts with the bootloader from Module 3.3. If your app is at an offset, the core would still try to fetch from address 0 -- unless someone tells it otherwise. Look up the **VTOR** register.)
3. List the exceptions Cortex-M0+ defines (NMI, HardFault, SVCall, PendSV, SysTick) and their vector table indices.

> Output of this module on paper: a numbered list of the vector table entries you will need in your startup file, in order. The first two are non-negotiable; the rest you can fill in as "unused handler that just spins" for now.

1.9 Module 6 -- Writing your own linker script

A linker script answers three questions:

1. **What memory regions exist?** (Name, origin address, length, allowed access.)
2. **Which output sections go where?** (`.text` to flash, `.data` to SRAM but loaded from flash, `.bss` to SRAM zero-initialized, `.stack`/`.heap` to SRAM.)
3. **What symbols does the startup code need?** (Where does `.data` live in flash? Where in RAM? Where does `.bss` start and end? Where is the top of stack?)

1.9.1 6.1 GNU ld linker script syntax -- a primer

The linker (`ld`, called via `arm-none-eabi-gcc -Wl,-T,linker.ld`) reads a linker script written in **GNU ld's own little language**. It's not C, it's not make -- it's its own thing. The full reference is `info ld` chapter "Scripts", but here are the building blocks you'll actually use.

1.9.1.1 File-level grammar

A linker script is a sequence of *commands* at the top level. Whitespace doesn't matter. Comments are C-style `/* ... */`. The handful of commands you'll actually write:

```
ENTRY(symbol)              /* sets the ELF entry-point symbol */
MEMORY { ... }             /* declares the physical memory regions */
SECTIONS { ... }           /* describes how input sections map to output */
PROVIDE(symbol = expr)     /* defines a symbol if nothing else has */
ASSERT(condition, "msg")   /* fails the link if condition is false */
INCLUDE filename           /* another script in-place */
```

1.9.1.2 MEMORY block

```
MEMORY
{
    name (attr) : ORIGIN = addr, LENGTH = size
    name (attr) : ORIGIN = addr, LENGTH = size
}
```

- `name` is what you'll refer to in `> NAME` clauses later (conventionally uppercase: `FLASH`, `RAM`).
- `attr` is a parenthesised string of access attributes: `r` readable, `w` writable, `x` executable, `a` allocatable, `!` inverts. Common choices: `(rx)` for flash, `(rwx)` for RAM. These aren't enforced by the chip -- they let the linker pick a default region for sections that don't say otherwise.
- `ORIGIN` and `LENGTH` accept normal integer expressions and the suffix multipliers `K` and `M` (powers of 1024).

Example skeleton for the MKR1000 (you fill in the numbers):

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00002000, LENGTH = 248K   /* example */
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 32K
}
```

The `FLASH` origin is offset by the bootloader size from Module 3.3. The example numbers above are illustrative -- confirm against your datasheet and the existing `flash_with_bootloader.ld`.

1.9.1.3 SECTIONS block

```
SECTIONS
{
    .output_section_name :
    {
        /* contents */
    } > REGION   AT> LOAD_REGION
}
```

Inside the braces you list **input section selectors**:

- `*(.text)` -- the `.text` section of every input object file.
- `*(.text*)` -- `.text` and any section starting with `.text` (e.g. `.text.foo` produced by `-ffunction-sections`).
- `file.o(.bss)` -- only from a specific object.

- `KEEP(*(.isr_vector))` -- like `*(...)` but exempt from `--gc-sections` garbage collection. Use `KEEP` for anything the linker can't see is referenced (vector table, startup code referenced only from the reset vector).

The trailing `> REGION` says: place the output section's **VMA** (Virtual Memory Address -- where it lives at runtime) in this region. The optional `AT> LOAD_REGION` says: place its **LMA** (Load Memory Address -- where the ELF loader / programmer puts the bytes) in a different region. This split is what makes initialized `.data` work: VMA in RAM (runtime address), LMA in flash (where the bytes are actually stored). Startup copies LMA -> VMA.

1.9.1.4 Location counter `.`

Inside a SECTIONS block, the symbol `.` is the **current address**. You can read it, assign to it (to pad/align), and use it in expressions. Setting it explicitly inside a region moves the next placement forward.

```
. = ALIGN(4);     /* round up to the next 4-byte boundary */
. = . + 0x400;    /* leave a 1 KiB hole */
```

`ALIGN(n)` returns its argument rounded up to a multiple of `n`. Many fundamental alignments (4 for words, 8 for double-words) are baked in by default but it's good practice to be explicit.

1.9.1.5 Defining symbols

You can assign to identifiers anywhere in a SECTIONS block. The right-hand side is an expression evaluated at link time; the symbol becomes part of the ELF symbol table and is visible to your C/asm code as an `extern`.

```
_sdata = .;               /* address of start of .data (VMA) */
*(.data*)                 /* contents */
_edata = .;               /* address of end of .data (VMA) */
_sidata = LOADADDR(.data); /* the LMA of .data, i.e. where bytes are in flash */
```

`LOADADDR(section)` is the built-in that asks "where is this section's LMA?" -- exactly what your startup needs to find the source of the copy.

A symbol defined like this from C:

```
extern uint32_t _sdata;       /* declares it */
uint32_t *dst = &_sdata;      /* &_sdata is the symbol's address, i.e. its value */
```

Subtle: a linker symbol has *no value of its own* -- its **address** is what matters. That's why you always use `&_sdata`, not `_sdata` directly.

1.9.1.6 A complete skeleton (annotated)

```
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00002000, LENGTH = 248K
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 32K
}

_estack = ORIGIN(RAM) + LENGTH(RAM);   /* top of stack, used by vector[0] */

SECTIONS
{
```

```
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))           /* the array of 32-bit vectors */
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
        *(.text*)                      /* all code */
        *(.rodata*)                    /* read-only data */
        . = ALIGN(4);
        _etext = .;                    /* end of code, start of .data's LMA */
    } > FLASH

    .data : AT (_etext)                /* VMA in RAM, LMA right after .text */
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
        _edata = .;
    } > RAM

    .bss :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)                      /* uninitialised globals without explicit section */
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    /* Optional: sanity-check that we haven't overrun RAM */
    ._user_heap_stack :
    {
        . = ALIGN(8);
        . = . + 0x400;                 /* at least 1 KiB of stack room */
    } > RAM
}
```

Read every line and explain it to yourself before you use it. The number-one mistake is copying a script with `AT (_etext)` from a tutorial without understanding that it places `.data`'s LMA at the location counter's value at that point -- which is fragile if you reorder sections.

1.9.1.7 Common error symptoms and what they mean

- **"section .text will not fit in region FLASH"** -- your code grew past LENGTH. Reduce `LENGTH` mistake or your binary is genuinely too big.

- **"section .data loaded at [a,b] overlaps section .text loaded at [c,d]"** -- your `AT` clause put `.data`'s LMA on top of `.text`. Use `AT> FLASH` (auto-place after current FLASH content) instead of `AT (_etext)`.
- **CPU faults instantly at boot, before main** -- your vector table's first word isn't a valid stack pointer (often because `_estack` is undefined or in the wrong region). Disassemble the start of your binary, confirm the first 4 bytes equal `ORIGIN(RAM) + LENGTH(RAM)`.
- **`memset` / `memcpy` silently doesn't happen** -- `--gc-sections` discarded your startup code because nothing references it. Wrap the vector table in `KEEP(...)` and make sure `ENTRY(Reset_Handler)` is set.

1.9.2 6.2 Memory regions

Write a `MEMORY { ... }` block with two regions, `FLASH` and `RAM`. The `ORIGIN` of FLASH is **not** the chip's flash base -- it's the post-bootloader offset from Module 3.3. The `LENGTH` shrinks correspondingly. RAM origin and length come straight from the datasheet's memory map.

1.9.3 6.3 Sections

Write a `SECTIONS { ... }` block. Minimum sections to handle:

- `.isr_vector` -- your vector table. **Must be the very first thing** in flash so it sits at the application's start address (which is where VTOR will point).
- `.text` -- code + read-only data.
- `.data` -- initialized RAM data, with LMA (load address) in flash and VMA (virtual/runtime address) in RAM. This is the `AT>` syntax. Define `_sidata`, `_sdata`, `_edata` symbols around it.
- `.bss` -- zero-initialized RAM. Define `_sbss`, `_ebss`.
- A stack region at the top of RAM. Define `_estack` (top of stack) as a symbol the vector table can use.

  Hint when stuck: read the gist linked in `README.md` ("Compiling baremetal C program for Arm Cortex-M0+") to understand the pattern, then close it and write your own from scratch -- copying defeats the point.

  Action: produce `linker.ld`. Test it by linking an empty program (`int main() { for(;;); }`) and inspect the result with `arm-none-eabi-objdump -h build/main.elf` and `arm-none-eabi-nm build/main.elf | sort`. Verify `_estack`, `_sdata`, `_edata`, `_sbss`, `_ebss`, `_sidata` exist and their addresses make sense.

1.10 Module 7 -- The startup file (`startup.s`) in ARM assembly

This is your `crt0` equivalent for the embedded world. Concept: **crt0** ("C runtime zero") is the glue between "the CPU just powered on" and "`main()` can run as if it were a normal C function." On a hosted system crt0 is provided by libc and does argv/argc/exit. On bare metal, you write it. It must:

1. Provide the vector table.
2. On reset: set the stack pointer, copy `.data` from flash to RAM, zero `.bss`, (optionally call C++ static constructors -- skip for now), call `main`, then spin if `main` returns.

1.10.1 7.1 Assembler basics for Cortex-M0+

- Cortex-M0+ runs **Thumb** instructions only. Use `.syntax unified` and `.thumb`.

- Each function: `.thumb_func` directive before the label, otherwise the linker won't set the low bit and the branch will fault.
- Use `.section .isr_vector, "a", %progbits` for the vector table; it has to land in the section your linker script placed first.

1.10.2 7.2 The vector table

It's just an array of 32-bit words. The **first word** is loaded into MSP by hardware at reset -- so it must be... (see Module 5 question 1, then put the right symbol from your linker script here). The **second word** is the address of your `Reset_Handler`. Words 3..N are the other exception vectors.

For now, point every exception except Reset at a `Default_Handler` that just does `b .` (infinite loop). You'll override individual ones later by giving them the same name with `.weak` attribute.

1.10.3 7.3 The Reset_Handler

Pseudocode of what it has to do, in order:

```
Reset_Handler:
    # 1. (Optional on M0+ since hardware already loaded MSP -- but be explicit
    #     if you want, or if you'll later relocate the vector table via VTOR.)
    # 2. Copy .data from flash (_sidata) to RAM (_sdata.._edata).
    # 3. Zero .bss (_sbss.._ebss).
    # 4. Call SystemInit (optional, for clock setup -- leave as weak stub for now).
    # 5. Call main.
    # 6. If main returns, infinite loop.
```

Each step is 4-8 Thumb instructions. The tricky parts:

- Loading a 32-bit symbol address into a register on Thumb-1 / M0+ uses `ldr Rn, =symbol` (literal pool). Look this up -- it's idiomatic.
- The copy and zero loops use `ldr`/`str` with post-increment patterns. M0+ has limited addressing modes compared to M3/M4 -- keep loops simple.

  Action: write `startup.s`. Link it together with your `main.c` and `linker.ld`. Disassemble with `arm-none-eabi-objdump -d build/main.elf` and **read every instruction**. If you can't explain why each one is there, you don't understand startup yet.

1.11 Module 8 -- specs files, libc, and what you actually need

1.11.1 8.1 What is a `.specs` file?

A specs file is a GCC "driver" config that adjusts which startup files, libraries, and include paths get added to the link line. The Arduino script uses two:

- `--specs=nano.specs` -- swaps full newlib for **newlib-nano** (smaller printf, etc.).
- `--specs=nosys.specs` -- provides empty implementations of OS syscalls (`_write`, `_sbrk`, `_exit`, ...) so libc links cleanly even though there's no OS.

You can write your own specs file. The format is documented in `man gcc` under `--specs=`. For learning, the most instructive specs file is one that says: "do not link the standard startup files, do not link the standard libraries, use my linker script."

1.11.2 8.2 Should you write your own libc?

Short answer: no, not for blink. Long answer: writing libc is a separate, large project. But the *useful* learning step is:

1. First, build **without any libc at all** (`-nostdlib -nostartfiles -nodefaultlibs`). Your blink does not need `memcpy`, `printf`, or `malloc`. Confirm you can link a bare program with zero external dependencies. This is the purest form of "bare metal" and is genuinely educational.
2. Later, when you want `memcpy`/`memset` (they're emitted implicitly by the compiler for struct copies and `.bss` clears even if you don't call them), reintroduce them: either write 10-line versions yourself in C, or link against newlib-nano with `--specs=nano.specs --specs=nosys.specs`.

   Action: in your Makefile, add `-nostdlib -nostartfiles -ffreestanding` to the link flags. Build. If the linker complains about undefined symbols like `memset`, write a minimal C version of just the ones it asks for.

1.11.3 8.3 Your own `mkr1000.specs`

Once the above works, encapsulate the flags into a specs file. Make it a hard requirement that `arm-none-eabi-gcc --specs=./mkr1000.specs ...` produces an identical binary to your previous direct-flags build. Compare with `sha256sum`.

1.12 Module 9 -- From "it links" to "the LED blinks"

Now you actually touch hardware. Don't open the datasheet at random; follow this order.

1.12.1 9.1 Find the LED

Action: open `docs/ABX00004/ABX00004-schematics.pdf`. Search for "LED" or "L" net. Find which **port and pin** of the SAMD21 it's connected to (e.g. PA20, PB10, ...). Also note: is it active-high or active-low (i.e. does driving the pin high or low light the LED)?

1.12.2 9.2 Clocks -- the SAMD21's reality check

Unlike an AVR, an ARM Cortex-M MCU has no peripheral life until you turn on its clock. On the SAMD21 this involves three subsystems you'll meet in the datasheet, in this order:

1. **GCLK** (Generic Clock Controller) -- routes clock sources to generators and from generators to peripherals.
2. **PM** (Power Manager) -- APB clock gates per peripheral. The PORT (GPIO) peripheral sits on one of the APB buses.
3. **SYSCTRL** -- controls the actual oscillators (OSC8M, DFLL48M, XOSC32K, ...).

For a first blink you can stay on the **OSC8M** internal 8 MHz oscillator that is already running at reset. You do **not** need to spin up the 48 MHz DFLL. Don't get sucked into PLL configuration for blink -- that's Module 11 material.

Action: in the datasheet, find: - Which APB bus is the PORT peripheral on? (APBA, APBB, or APBC?) - Which bit of which PM register enables the clock to PORT? - Is the PORT clock enabled out of reset? (Check the reset values table; if yes, you can skip enabling it.)

1.12.3 9.3 GPIO -- the PORT peripheral

> Action: open the **PORT** chapter. Find: - Base address of `PORT` and the layout of one group (each port A/B is one "group" with registers `DIR`, `OUT`, `OUTSET`, `OUTCLR`, `OUTTGL`, `PINCFG[]`, ...). - The minimum sequence of register writes to (a) configure pin Pxn as a digital output, (b) drive it high, (c) drive it low.

A minimal sequence is roughly: set the corresponding bit in `DIRSET`, then write `OUTSET`/`OUTCLR` to drive the level. The `PINCFG[n]` register usually does not need touching for a plain output. Verify all of this in the datasheet -- don't trust this paragraph.

1.12.4 9.4 Delay (the bad way, on purpose)

For a first blink, a software delay loop is fine:

```
for (volatile uint32_t i = 0; i < 200000; ++i) { }
```

Mark the counter `volatile` so `-O2` doesn't delete the loop. Yes, this is awful engineering -- but it lets you verify the LED works before you tackle SysTick. SysTick is the next module.

1.12.5 9.5 Put it together

Your `main.c` becomes:

```
int main(void) {
    // 1. (optional) enable PORT clock if not on by default
    // 2. configure LED pin as output
    // 3. loop: toggle LED, delay
}
```

> Action: build, `objcopy` to `.bin`, flash with bossac, watch the LED. If it doesn't blink: - Did the bootloader actually hand off? (Did your binary write to the right flash offset?) - Is the pin really the LED pin? (Re-check schematic.) - Is the polarity right? - Is the clock to PORT actually enabled? - Use `arm-none-eabi-objdump -d` to confirm your `main` is at the address your vector table's reset vector points to.

1.13 Module 10 -- An easy peripheral without a HAL: SysTick

You've now driven GPIO directly through registers. Time to do the same with a real peripheral. **SysTick is the easiest one** and gives you a proper delay function as a bonus.

1.13.1 Why SysTick first

- It's part of the **Cortex-M0+ core**, not a SAMD21 peripheral. That means:
  - It works without any clock setup beyond what the core already has (its input is the CPU clock by default).
  - It's documented in the Cortex-M0+ Generic User Guide, not the SAMD21 datasheet -- a much shorter chapter.
  - You'll find the same SysTick on every Cortex-M MCU you ever touch.
- It has **only four registers** in the PPB region (`0xE000E010-0xE000E01C`): CTRL, LOAD, VAL, CALIB.
- It generates an exception you've already reserved a slot for in your vector table (Module 5).

1.13.2 What it does

SysTick is a 24-bit down-counter. You load it with a reload value, start it, and on every tick it decrements. When it hits zero it (a) sets a COUNTFLAG bit in CTRL and (b) optionally generates the SysTick exception. Then it reloads and continues. That's the entire peripheral.

1.13.3 Practical limits of a 24-bit counter

The reload register is 24 bits, so the longest single-shot interval is `(2^24 - 1) / f_cpu`. At the boot-default 8 MHz that is about 2.1 s; at 48 MHz (Module 11) it shrinks to about 350 ms. For longer intervals you **must** run SysTick at a small tick (1 ms is conventional) and count ticks in software -- which is what the interrupt-driven pattern below does.

1.13.4 What to figure out

> Action: in the Cortex-M0+ user guide's "System Timer (SysTick)" chapter, find: - The four register names, addresses, and bit fields. - What the CTRL bits do: `ENABLE`, `TICKINT`, `CLKSOURCE`. (Note: on Cortex-M0+, the external/reference clock option may or may not be implemented depending on the chip -- check the SAMD21 datasheet's "Implementation Defined Behaviour" section if there is one. If unsure, leave `CLKSOURCE = 1` for processor clock.) - The maximum reload value (it's 24-bit, so 2^24 - 1). - For a system clock of 8 MHz (OSC8M), what reload value gives a 1 ms tick?

1.13.5 Two ways to use SysTick

Polling style (no interrupts, simplest):

```
// Pseudocode -- fill in actual register addresses from the user guide.
void delay_ms(uint32_t ms) {
    // For each ms, load reload, clear current, start, wait for COUNTFLAG, stop.
    // Or: load with (cpu_freq/1000 - 1), start once, count COUNTFLAG sets.
}
```

Interrupt style (proper tick):

```
volatile uint32_t systick_ms;

void SysTick_Handler(void) {  // matches the name in your vector table
    systick_ms++;
}

void delay_ms(uint32_t ms) {
    uint32_t start = systick_ms;
    while ((systick_ms - start) < ms) { /* WFI to save power */ }
}
```

For the interrupt version: rename your `Default_Handler` slot for SysTick in the vector table to `SysTick_Handler` (or use `.weak` aliases). Also: SysTick is a core exception, **not** an NVIC interrupt -- you don't enable it in NVIC, you just set TICKINT in SysTick's own CTRL register.

1.13.6 A note on power

The `WFI` ("Wait For Interrupt") instruction puts the core to sleep until the next interrupt fires. With a SysTick interrupt at 1 ms, looping `WFI` in your idle path means the CPU is awake for only a tiny fraction

of each tick. On battery-powered designs this is the difference between days and weeks of runtime. Inline it as `__asm volatile ("wfi");` or use the `__WFI()` intrinsic. Deeper sleep modes (STANDBY, BACKUP) are a separate subject in the SAMD21 "PM" chapter -- not needed for blink, very relevant if you ever battery-power the MKR1000.

> Action: replace your software-delay blink with a SysTick-driven blink. Vary the rate. If LED period is wrong by a factor of N, your assumed CPU clock is wrong by N -- useful sanity check before you move to Module 11.

1.13.7 A note on other "easy" peripherals to try next

After SysTick, in roughly increasing complexity:

1. **EIC (External Interrupt Controller)** -- read a button via interrupt. Same register-poking style as PORT.
2. **TC (Timer/Counter) in basic 16-bit timer mode** -- generate a PWM on the LED to fade it. Requires GCLK setup.
3. **ADC** -- read a potentiometer voltage. Significant setup but well-documented.
4. **SERCOM in UART mode** -- see Module 11 below.
5. **SERCOM in SPI mode** -- needed for the WiFi chip (Module 14).
6. **USB device** -- months of work, only attempt after Module 11.

Resist building a "HAL" until you've poked the registers of at least three of these peripherals by hand. Premature HAL is just untested abstraction with no shape.

1.14 Module 11 -- Real clocks and a UART

This module is the bridge to the WiFi work in Module 14. You need:

1. **48 MHz CPU clock** via the DFLL48M. The MKR1000 has a 32.768 kHz crystal (XOSC32K) -- find it on the schematic. The standard recipe is: `XOSC32K -> GCLK1 -> DFLL48M reference -> DFLL48M in closed-loop mode -> GCLK0 -> CPU/AHB/APB`. Each arrow is a small register dance.
2. **Read DFLL coarse calibration from the NVM Calibration Row** (Module 4 item 4) before starting the DFLL -- the datasheet explicitly recommends this to get an accurate 48 MHz quickly.
3. **NVMCTRL wait states**: at 48 MHz the flash needs **1 wait state**. Set it in `NVMCTRL.CTRLB.RWS` *before* you switch the clock, or you'll start reading garbage from flash mid-instruction.
4. **A SERCOM in UART mode** so you can `printf` over the MKR1000's pre-broken-out TX/RX pins. The Arduino "Serial1" uses one of SERCOM5's pads; find which pads on the schematic. UART baud rate register on SAMD21 uses a fractional formula -- look it up in the SERCOM-USART chapter.

   > Action: get `printf("hello\r\n")` over UART running. Wire a USB-UART adapter (CP2102/CH340/FT232) to the MKR1000's TX pin and ground, open `picocom` or `screen` at your chosen baud rate. This is your debug lifeline for everything after this.

For `printf` itself: link newlib-nano and implement `_write` to push to your UART. This is a 10-line function. (You may also need stubs `_sbrk`, `_close`, `_lseek`, `_read`, `_fstat`, `_isatty` -- most can return -1.)

1.15 Module 12 -- Debug probes: JTAG, SWD, and using one

You can do this whole course up to Module 11 with just USB + bossac. This section exists so you understand what you are **not** using, and what you'd gain by adding a probe later.

1.15.1 12.1 The two ways to get code onto an MCU

1. **Through a bootloader already in flash** (what you're doing now). The MCU runs a small program that listens on USB/UART and writes the rest of flash for you. Pro: no extra hardware. Con: the bootloader itself has to be there -- brick it and you're stuck without option 2. Also, you can't single-step debug through this channel.
2. **Through the CPU's debug port**, using external hardware that physically wiggles dedicated pins on the chip. This works even with **completely blank flash**, lets you debug, and is how the bootloader got there in the first place.

1.15.2 12.2 JTAG vs SWD

Both are debug interfaces defined by ARM (well, JTAG predates ARM; ARM adopted it).

- **JTAG**: 4-5 wires (TCK, TMS, TDI, TDO, optional TRST). Older, more general (originally for boundary-scan testing of PCBs), supports daisy-chaining multiple chips.
- **SWD** (Serial Wire Debug): 2 wires (SWCLK, SWDIO) + ground + reference voltage. ARM-specific. Functionally equivalent to JTAG for debugging a single chip. **Cortex-M0+ implementations typically expose SWD only**, not JTAG. The SAMD21 is in this category -- confirm by searching the SAMD21 datasheet for "SWD" and "JTAG".

You don't choose between them; the chip chooses for you. On the MKR1000 you get SWD.

> Action: in `docs/ABX00004/ABX00004-schematics.pdf`, find the SWD test points (look for nets named `SWCLK`, `SWDIO`, and possibly `RESET`). Note whether they're broken out to header pads or only to test points you'd need to solder to.

1.15.3 12.3 What a "probe" or "programmer" actually is

A debug probe is a small USB device that translates **USB packets to SWD/JTAG wiggling**. On the host side it speaks some protocol that OpenOCD/pyOCD/Segger software understands; on the target side it drives SWCLK/SWDIO. That's it.

Common probes you'd actually encounter:

Probe	Protocol on USB side	Open?	Typical price	Notes
ST-Link/V2 (often a clone)	ST proprietary	Closed firmware, but well reverse-engineered; OpenOCD supports it	very cheap	Officially for STM32, but clones happily debug any SWD target. Cheapest reasonable entry.
J-Link (Segger)	Segger proprietary	Closed	high (educational version cheaper)	Gold standard for speed and reliability. Overkill for hobby.

Probe	Protocol on USB side	Open?	Typical price	Notes
CMSIS-DAP	Standardized by ARM	**Open**	varies	Many implementations. Most importantly:
Raspberry Pi Pico as "debugprobe"	CMSIS-DAP	Fully open	a few EUR	A Pi Pico flashed with Raspberry Pi's `debugprobe` firmware *is* a CMSIS-DAP probe. Two wires to your target. Recommended if/when you want to add SWD.

The MKR1000 itself does **not** have an on-board debugger (unlike the SAM-W25 Xplained Pro development board you have docs for, which does). So you cannot self-debug it without external hardware.

1.15.4 12.4 What you'd actually do with a probe

1. **Flash without the bootloader** -- write your app to flash address 0 directly, freeing up the space the bootloader occupies and letting you change your linker script's FLASH origin to the chip's true flash base.
2. **Recover a bricked board** -- if you overwrite the bootloader or wedge the CPU, only SWD can get you back.
3. **Single-step debugging with GDB**:

   ```
   openocd -f interface/cmsis-dap.cfg -f target/at91samdXX.cfg
   # in another terminal:
   arm-none-eabi-gdb build/main.elf
   (gdb) target extended-remote :3333
   (gdb) load
   (gdb) break main
   (gdb) continue
   ```

   Breakpoints, register inspection, live memory. Once you've debugged one bug this way you'll wonder how you lived without it.
4. **Read/write the chip's fuses and lock bits**, which the bootloader interface doesn't expose.

1.15.5 12.5 Software stack on the host

- **OpenOCD**: the open-source universal debug server. Speaks to nearly every probe on one side and exposes a GDB server + telnet command interface on the other.
- **pyOCD**: pure-Python alternative, simpler config, narrower hardware support. Good for CMSIS-DAP specifically.
- **GDB** (`arm-none-eabi-gdb`, comes with your toolchain): the actual debugger. Connects to OpenOCD/pyOCD over TCP.

You do not need any of this for blink. Note it down as "Module 12 territory" and continue with bossac.

1.15.6 12.6 Recommendation for you, given USB-only right now

- Stick with bossac for the whole blink-and-beyond course up through Module 11.
- **The moment** you decide to (a) reclaim the bootloader's flash space, (b) debug a hang you can't `printf` your way out of, or (c) write your own bootloader (Module 13) -- get a Raspberry Pi Pico and flash it with the Raspberry Pi `debugprobe` firmware. Wire SWCLK/SWDIO/GND to the MKR1000 test points. Use OpenOCD.

1.15.7 12.7 Hands-on: getting a probe and using it

When you're ready (after blink, ideally after UART):

1. **Get a Raspberry Pi Pico** (cheap). Two of them is even better -- one is the probe, one is a target for practice if you ever brick the MKR1000.
2. **Flash it with `debugprobe`**: download the prebuilt UF2 from `raspberrypi/debugprobe` releases, hold BOOTSEL, plug into USB, drag-and-drop the UF2. Done.
3. **Wire it to the MKR1000**:
   - Pico GP2 -> MKR1000 SWCLK
   - Pico GP3 -> MKR1000 SWDIO
   - Pico GND -> MKR1000 GND
   - Optionally Pico GP4/GP5 -> MKR1000 UART for combined debug + serial.
4. **Install OpenOCD** on Void (verify package name with `xbps-query -Rs openocd`).
5. **First connection**:

   ```
   openocd -f interface/cmsis-dap.cfg -f target/at91samdXX.cfg -c "adapter speed 2000"
   ```

   You should see "Cortex-M0+ ... running" or similar. If you do, you can now read/write any byte of memory on the chip and the bootloader's monopoly is broken.
6. **First debug session**: rebuild your blink with `-g -O0`, in one terminal run OpenOCD, in another run `arm-none-eabi-gdb build/main.elf`, then `target extended-remote :3333`, `monitor reset halt`, `load`, `break main`, `continue`. Step through your own startup code instruction by instruction. **You will learn more in this one session than in the previous three modules combined.**
7. Once that works: try erasing the bootloader (`monitor flash erase_sector 0 0 1` or similar -- confirm syntax) and reflashing your app to flash address 0 with a modified linker script. This proves you've escaped the bootloader entirely.

1.16 Module 13 -- The bootloader: understand it, then write your own

This is a serious project -- comparable in size to everything before it combined -- but extremely rewarding. Tackle it only after Module 11.

1.16.1 13.1 What a bootloader actually is

A bootloader is just **a normal application** that happens to be placed at the address the CPU jumps to after reset, and whose job is to put **a second application** somewhere in flash and then transfer control to it. There's nothing magic about it -- same Cortex-M0+ startup, same vector table, same linker script discipline.

What makes a bootloader interesting are the design questions:

1. Where does my application live, and how does the bootloader know where to jump?
2. How does the bootloader decide "should I run the app, or should I stay in bootloader mode and accept new firmware?" (The double-tap-reset trick on Arduino boards is one answer.)
3. What transport accepts the new firmware? (USB CDC? UART? CAN?)
4. What protocol does it speak? (SAM-BA? XMODEM? Custom?)
5. How do you avoid bricking the board if the user pulls power mid-flash?

1.16.2 13.2 Study the existing one first

Before writing your own, **read** the existing MKR1000 bootloader. Sources:

- **The Arduino SAMD bootloader source**: it's open source. Repo `arduino/ArduinoCore-samd` on GitHub, under `bootloaders/zero/` (the MKR1000 uses a variant of the Arduino Zero bootloader). It's a few thousand lines of C -- small enough to read end to end in a weekend.
- **SAM-BA protocol documentation**: search for "Atmel SAM-BA Boot Assistant" application notes. The protocol is text-based over serial and surprisingly simple (commands like `N#` for version, `W,addr,value#` for write word).

  Action: clone `arduino/ArduinoCore-samd`, navigate to `bootloaders/zero/`, and read at minimum: - `main.c` -- the entry point. Look for the "should I stay in bootloader?" decision and find how it's signaled. - The linker script -- note that the bootloader's linker script places it at flash base, and your application's linker script places the app *after* the bootloader. Two halves of the same agreement. - The USB CDC stack -- this is the heaviest part. USB on the SAMD21 is non-trivial.

  Write a one-page summary in your own words of how it boots. You're now qualified to write your own.

1.16.3 13.3 The double-tap trick

A juicy puzzle to chew on before reading the answer:

> Question: SRAM contents are not guaranteed to survive a reset, but the bootloader uses a magic value in SRAM to detect "the user just tapped reset twice within 500 ms, enter bootloader mode." How is this possible?

Hints in order of how much they spoil:

1. SRAM contents are not *cleared* by a CPU reset on this chip -- they're just *undefined* on power-up. A reset that doesn't cycle power leaves them intact (see Module 4).
2. The bootloader writes a magic value at a fixed SRAM address, then waits a short time. If the user resets again within that window, the bootloader (now running again) sees the magic value still there -> stay in bootloader mode. If no reset comes, the bootloader clears the magic and jumps to the app.
3. Your application's startup code must therefore **not** clear that one specific word of SRAM during its `.bss` zeroing -- or alternatively, the magic word lives in a special section that startup skips.

Action: find the magic value and its address in the Arduino bootloader source. Find how the application avoids clobbering it (or how the bootloader clears it before jumping, making the application's job irrelevant).

1.16.4 13.4 Jumping from bootloader to application

The handoff is conceptually simple:

```
Application's vector table is at address APP_START.
The first word at APP_START is the application's initial stack pointer.
The second word at APP_START is the application's reset handler.

To jump:
  1. Disable interrupts.
  2. Set VTOR (Vector Table Offset Register) to APP_START so the app's
     vector table is used for interrupts.
  3. Load the new MSP from *(uint32_t*)APP_START.
  4. Branch to *(uint32_t*)(APP_START + 4).
```

This is around six lines of C with some inline assembly. Practical gotchas:

- Disable peripherals the bootloader enabled (USB, clocks) -- or the application will inherit unexpected state.
- VTOR on Cortex-M0+ has alignment constraints. Look up the exact requirement in `dui0662a`.

1.16.5 13.5 Writing your own -- staged plan

Do this as a separate course, after Module 11. Don't attempt all at once.

Stage 1: smallest possible bootloader (UART, no USB). - Linker script places it at flash base, length = some small amount (start with 8 KiB to mirror Arduino's). - On boot, set up UART, print a prompt. - Accept a simple command: "write N bytes starting at address A". Use Intel HEX or XMODEM as the format (XMODEM is famously simple -- around 100 lines). - After a command "go", jump to APP_START as described above. - Build a tiny app that just blinks, linked at APP_START, flash it via your bootloader, watch it blink. **This is a massive milestone.**

Stage 2: USB CDC bootloader. - This requires implementing (or porting) a USB device stack. The SAMD21 USB peripheral is documented in the datasheet but it's a thicket. Plan for a multi-week project. - Speak SAM-BA protocol so existing `bossac` works against your bootloader. (Or define your own protocol and write your own host tool -- also very educational.)

Stage 3: survive partial flashes. - A reset partway through writing the app must not brick the board. - Standard technique: a "valid app" flag stored in flash (or in NVM user row) that the bootloader writes only *after* the entire app has been written and verified. If the flag is missing, stay in bootloader mode.

1.16.6 13.6 Pre-requisite: an SWD probe

Do not start writing your own bootloader without an SWD probe in hand. The moment your bootloader has a bug -- and it will -- your only recovery path through USB is gone. With SWD you wipe flash and try again in 10 seconds. Without it, you've bricked the board until you get a probe anyway. Same applies to any experiment that modifies the existing bootloader.

This is the natural pairing: Module 12 (get a probe) and Module 13 (write a bootloader) belong together.

1.17 Module 14 -- A HAL design proposal and the road to a WiFi server

By the time you reach this module you should have, from your own code: GPIO, SysTick, UART over SERCOM, 48 MHz clock. Now we structure the next big step.

1.17.1 14.0 Register-access hazards: volatile and memory barriers

You've been writing `volatile` on register pointers since Module 4 without much justification. Before you start driving an SPI peripheral and a separate chip across it, get explicit about what the qualifier does and where it isn't enough.

`volatile` tells the compiler: every read and every write of this object is observable, must happen in source order, and cannot be combined, removed, or reordered relative to other `volatile` accesses. That's it. It does *not*:

- Order accesses relative to **non-volatile** memory (the compiler can move them past `volatile` ops).
- Prevent the **CPU's bus interface** from reordering writes, completing them out of order, or letting a later read overtake an earlier write.
- Make accesses **atomic** across interrupts or DMA.

On Cortex-M0+ the CPU itself is single-issue and in-order, but the **peripheral bus** still has buffering and posted writes. Symptoms when this bites you:

- You enable a peripheral's clock in GCLK, then immediately write its registers, and the write is silently dropped because the clock hadn't propagated yet.
- You set VTOR and immediately enable interrupts (bootloader-to-app handoff); the first interrupt fetches a vector from the *old* table.
- You write to flash via NVMCTRL, then read back and get stale data.

The fix is a **memory barrier instruction**. On Cortex-M:

Instruction	Effect
`DMB` (Data Memory Barrier)	Earlier explicit memory accesses complete before later ones start.
`DSB` (Data Synchronisation Barrier)	Stronger: also drains pending writes; nothing after `DSB` executes until earlier memory ops are done.
`ISB` (Instruction Synchronisation Barrier)	Flushes the prefetch pipeline; later instructions are refetched. Required after VTOR change or after enabling/disabling features that change instruction execution.

In C, use the compiler intrinsics: `__DMB()`, `__DSB()`, `__ISB()` (from `cmsis_gcc.h` if you adopt CMSIS, or write your own one-line inline asm: `__asm volatile ("dmb 0xF" ::: "memory");`).

> Rules of thumb for this project: - After enabling a peripheral's clock in `PM`/`GCLK`, issue a `DSB` before touching the peripheral's registers. - After writing `SCB->VTOR`, issue `DSB; ISB;` before enabling interrupts. - In your bootloader-to-app jump (Module 13.4), the sequence is: write VTOR, `DSB`, set MSP, `ISB`, branch. Skipping the barriers works "most of the time" -- which is the worst kind of bug. - Between bytes of an SPI transfer where the CS line is held by software, `DSB` ensures the byte is fully out before you toggle CS.

For *interrupt vs. main-thread* shared state (e.g. the `systick_ms` counter in Module 10), `volatile` alone is sufficient on Cortex-M0+ *for naturally- aligned 32-bit reads and writes* because those are atomic by

architecture. For anything wider or unaligned, disable interrupts around the access.

1.17.2 14.1 What the WiFi side actually looks like on this board

The MKR1000's WiFi is **not** the SAMD21. The SAMD21 only speaks to the **ATWINC1500** WiFi controller (a separate IC inside the ATSAMW25 module) over **SPI**, plus a few GPIO signals: chip select, reset, IRQ from WINC to MCU, and optionally a chip-enable. The WINC1500 has its own ARM CPU, its own firmware, and its own TCP/IP stack on-chip.

> Action: open `docs/ATSAMW25-MR210PB/Atmel-42618-...Datasheet.pdf` and `docs/ATSAMW25-MR210PB/Atmel-` From these and `docs/ABX00004/ABX00004-schematics.pdf` find: - Which SAMD21 SERCOM is wired to the WINC1500 SPI bus, and on which pads. - The exact SAMD21 pins for: WINC SPI CS, WINC RESET, WINC IRQ, WINC chip enable / EN. Note their polarities. - The maximum SPI clock frequency the WINC1500 will accept (relevant for SERCOM SPI baud register configuration).

This means writing a WiFi server breaks into two layers:

1. **A SERCOM SPI driver** on the SAMD21 (your code).
2. **A WINC1500 host driver** that runs on the SAMD21 and uses your SPI driver to talk to the WINC1500 firmware. Microchip publishes their WINC1500 Host Driver as **open-source C**, designed exactly for this porting scenario -- you supply a small platform layer (SPI transfer, GPIO control, delay, interrupt hook) and they provide the rest, including the socket API.

Trying to talk to the WINC1500 with your own from-scratch protocol stack is a PhD project. Use Microchip's driver -- porting it *is* the educational work, and it's the work that produces a working WiFi server in finite time.

1.17.3 14.2 A HAL proposal

The aim of your HAL should be: **exactly the layer the WINC1500 host driver needs, plus what your application uses for I/O. No more.**

Suggested module layout (one `.c` + one `.h` per module):

```
src/
  startup.s            # already from Module 7
  main.c
hal/
  clock.{c,h}          # init_clocks_48mhz(), get_cpu_freq()
  gpio.{c,h}           # pin config, set, clear, toggle, read, attach-interrupt
  systick.{c,h}        # systick_init(), millis(), delay_ms()
  uart.{c,h}           # uart_init(baud), uart_putc(), uart_write(), printf hookup
  spi.{c,h}            # spi_init(sercom, baud, mode), spi_transfer(byte), spi_select(pin)
  nvic.{c,h}           # small wrappers: enable_irq, set_priority
include/
  samd21.h             # only the registers you actually use, hand-defined
winc/                  # Microchip's WINC1500 host driver, vendored
  bsp/
    bsp_samd21.c       # the platform layer that uses your hal/ to drive the WINC
  driver/
    ...                # untouched from Microchip
```

Design principles for your HAL:

- **No dynamic allocation.** Every HAL object is statically defined.
- **One config struct per init function**, passed by `const` pointer. Easy to see what's configurable, easy to grep.
- **Blocking by default**, non-blocking variants suffixed `_async`. Don't invent a callback framework before you need it.
- **No premature abstraction over peripherals.** Don't try to make `uart_init` generic over "any UART-like thing." Just configure SERCOMn-as-UART.
- **Errors as small enums returned from functions.** No exceptions, no global errno.
- **Keep the HAL freestanding** (no libc deps beyond `stdint.h`, `stdbool.h`, and any `mem*` you've written yourself or pulled from newlib-nano).
- **Volatile correctness is in `samd21.h`**, not in every HAL function. Each register macro is `(*(volatile uint32_t*)0xADDRESS)`. Then the HAL bodies read normally.

If at any point you're tempted to add a feature "just in case" -- don't. Add it when the WINC1500 BSP or your application asks for it.

1.17.4 14.3 The road to "TCP server replies to a browser"

Staged milestones, each independently verifiable:

1. **SPI loopback.** Wire SERCOM MOSI to MISO with a jumper. `spi_transfer(0xA5)` returns `0xA5`. (Independent of the WINC1500 -- this proves your SPI driver.)
2. **WINC1500 power-up and chip-ID read.** Implement the minimum BSP (SPI transfer + reset GPIO + delay). Drive RESET low, wait, drive RESET high, wait per the WINC datasheet, then issue the WINC's "read chip ID" register command. Print the result over UART. A non-zero, datasheet-matching ID is massive progress.
3. **WINC1500 firmware version.** Use the host driver's `m2m_wifi_init()` and ask for firmware version. This requires the WINC1500's firmware to already be on the chip -- the MKR1000 ships with it, but version may be old. (Note: updating WINC firmware is itself a project -- use Arduino's WiFi101 FirmwareUpdater as a fallback if needed, or skip until later.)
4. **Connect to your WiFi access point.** `m2m_wifi_connect()` with SSID + password. Print "connected" callback fires. You now have a station.
5. **TCP socket server.** Open a listening socket on port 80 with the host driver's `socket()`/`bind()`/`listen()`/`accep` API. On `accept`, read request, write a hardcoded HTTP/1.1 response: `HTTP/1.1 200 OK\r\nContent- Length: 13\r\n\r\nHello, world!`. Point your browser at the MKR1000's IP.
6. **Make the response dynamic.** Toggle the LED based on a `GET /led/on` vs. `GET /led/off`. You've now closed the loop from WiFi -> SAMD21 -> GPIO and you have a functioning embedded web server with no Arduino code anywhere in your build.

1.17.5 14.4 What if you want to go further than the on-chip stack?

Optional, ambitious: the WINC1500 has a **bypass mode** where it acts as a plain layer-2 WiFi link and you run your own TCP/IP stack (typically **lwIP**) on the SAMD21. This is a much larger project and you don't need it for a working server -- but it's the path if you want to write your own TCP stack someday.

For learning, the recommended order is:

1. Get the on-chip-stack web server working (steps 1-6 above). **Stop here for v1.**
2. Only then consider bypass mode + lwIP, as a separate course.

1.18 Suggested directory layout to grow into

```
mkr1000/
+-- Makefile
+-- linker.ld          # Module 6
+-- mkr1000.specs      # Module 8.3
+-- startup.s          # Module 7
+-- src/
|   +-- main.c         # Module 9+
+-- hal/               # Module 14
|   +-- clock.{c,h}
|   +-- gpio.{c,h}
|   +-- systick.{c,h}
|   +-- uart.{c,h}
|   +-- spi.{c,h}
+-- include/
|   +-- samd21.h       # register #defines, written as you need them
+-- winc/              # Module 14 (later)
    +-- bsp/
    +-- driver/
```

Resist the urge to download a giant CMSIS `samd21.h` header. Define the registers you actually use, by hand, as `#define REG (*(volatile uint32_t*)0xADDRESS)`. After a few peripherals you'll deeply understand what CMSIS is and *then* you can decide whether to adopt it.

1.19 How to use this guide

- **Read Module 0 first** as orientation. It's the only "what hardware am I holding?" reading in this guide; the rest assumes you know.
- Work strictly in order through Module 11. The dependencies are real.
- Modules 10 (SysTick) and 11 (clocks + UART) can technically be done in either order after Module 9, but doing SysTick first gives you a sanity check on the default 8 MHz clock before you change it.
- Module 13 (write a bootloader) requires Module 12 (get a probe) first, for safety. Don't reverse that order.
- Module 14 (HAL + WiFi) requires Modules 9, 10, 11. It does not require 12 or 13 -- you can have a working WiFi server without ever owning a probe, as long as you keep the stock bootloader intact.

When you get stuck on a module, the answer is almost always in one of the three documents at the top of this file. If it's not there, then search the web -- but check the date and prefer Microchip/ARM primary sources over blog tutorials.

1.20 Appendix A -- Glossary

Terms and acronyms used in this guide, plus a few you'll meet immediately when you start reading datasheets.

1.20.1 Architecture / CPU

- **ABI** -- Application Binary Interface. Calling convention, register usage, stack layout. `eabi` is ARM's embedded ABI.
- **AAPCS** -- Procedure Call Standard for the ARM Architecture. The specific rules under EABI: r0-r3 for args, r0(-r1) for return, r4-r11 callee-saved, etc.
- **AHB** -- Advanced High-performance Bus. The fast on-chip bus on ARM SoCs, used between CPU, SRAM, and high-bandwidth peripherals.
- **APB** -- Advanced Peripheral Bus. Slower bus for low-bandwidth peripherals. SAMD21 has APBA, APBB, APBC.
- **ARM** -- both the company and the architecture they design.
- **ARMv6-M / ARMv7-M / ARMv8-M** -- instruction-set architecture profiles for Cortex-M cores. Your M0+ is ARMv6-M.
- **Cortex-A / -R / -M** -- ARM's three core families: Applications, Real-time, Microcontroller.
- **EABI** -- Embedded ABI. The "abi" in `arm-none-eabi`.
- **FPU** -- Floating-Point Unit. Optional on M4/M7/M33/M55/M85; absent on M0+.
- **Harvard architecture** -- separate code and data address spaces. AVR is Harvard; Cortex-M is not.
- **Instruction set / ISA** -- Instruction Set Architecture. The vocabulary of machine instructions the CPU understands.
- **MMU** -- Memory Management Unit. Hardware that translates virtual to physical addresses. Cortex-A has one; Cortex-M doesn't.
- **MPU** -- Memory Protection Unit. Simpler than an MMU; enforces access rules on regions without virtualising addresses. Optional on Cortex-M.
- **MSP** -- Main Stack Pointer. The default stack pointer used by Cortex-M after reset.
- **NVIC** -- Nested Vectored Interrupt Controller. The Cortex-M interrupt controller. Lives in the PPB region.
- **PPB** -- Private Peripheral Bus. Memory region `0xE0000000+` containing the core's own registers (SysTick, NVIC, SCB).
- **PSP** -- Process Stack Pointer. Alternate Cortex-M stack pointer, used by RTOSes for thread stacks.
- **RISC** -- Reduced Instruction Set Computer. Few, simple, fixed-width instructions. ARM and RISC-V are RISC.
- **CISC** -- Complex Instruction Set Computer. Many variable-width instructions, often with memory operands. x86 is CISC.
- **SCB** -- System Control Block. Cortex-M block containing VTOR, AIRCR, etc.
- **SysTick** -- the 24-bit core timer inside every Cortex-M.
- **Thumb / Thumb-2** -- ARM's 16-bit (Thumb) and mixed 16/32-bit (Thumb-2) instruction encodings. Cortex-M runs Thumb only.
- **TrustZone-M** -- security extension in ARMv8-M (M23/M33/M55/M85).
- **VTOR** -- Vector Table Offset Register. Tells the core where the vector table is located.

1.20.2 SAMD21 / Microchip-specific

- **APBA/APBB/APBC** -- the three APB buses on the SAMD21.
- **DFLL48M** -- Digital Frequency Locked Loop, 48 MHz output. Generates the CPU clock when running at 48 MHz.
- **EIC** -- External Interrupt Controller.
- **GCLK** -- Generic Clock Controller. Routes clock sources to peripherals.
- **NVMCTRL** -- Non-Volatile Memory Controller. Programs flash and the NVM user row.
- **NVM** -- Non-Volatile Memory. On SAMD21, refers to flash plus the user row and calibration row.
- **OSC8M** -- Internal 8 MHz oscillator, running by default at reset.
- **PM** -- Power Manager. Holds APB clock enables and reset controls.

- **PORT** -- the GPIO peripheral.
- **SERCOM** -- Serial Communication peripheral. Configurable as UART, SPI, or I2C. SAMD21 has six (SERCOM0-5).
- **SYSCTRL** -- System Controller. Manages oscillators (OSC8M, DFLL, XOSC).
- **TC / TCC** -- Timer/Counter and Timer/Counter for Control.
- **WDT** -- Watchdog Timer.
- **XOSC32K** -- External 32.768 kHz crystal oscillator (the MKR1000 has one).

1.20.3 Memory / linking

- **BSS** -- "Block Started by Symbol". The section for uninitialised globals, zeroed at startup.
- **LMA** -- Load Memory Address. Where bytes are stored at rest (typically flash for `.data`).
- **VMA** -- Virtual Memory Address. Where bytes live at runtime (RAM for `.data`, even though there's no virtual memory).
- **.text** -- code and read-only data section.
- **.rodata** -- read-only data section (sometimes part of `.text`).
- **.data** -- initialised read/write data section.
- **.bss** -- uninitialised read/write data section.
- **crt0** -- C runtime 0. The startup glue between reset and `main`.
- **ELF** -- Executable and Linkable Format. The output of the linker.
- **HEX** -- Intel HEX. Text-format firmware file.
- **BIN** -- raw binary, no headers.
- **specs file** -- GCC config that modifies what gets added to compile/link lines.

1.20.4 Tools / debugging

- **bossac** -- open-source flasher that speaks SAM-BA over USB CDC.
- **CMSIS** -- Cortex Microcontroller Software Interface Standard. ARM-defined C headers and abstractions for Cortex-M.
- **CMSIS-DAP** -- open standard for USB debug probes.
- **CDC** -- Communications Device Class. USB device class for virtual serial ports.
- **GDB** -- GNU Debugger.
- **JTAG** -- Joint Test Action Group. Original ARM debug interface, 4-5 wires.
- **OpenOCD** -- Open On-Chip Debugger. Open-source debug server.
- **pyOCD** -- Python-based debug server for CMSIS-DAP.
- **SAM-BA** -- Atmel/Microchip's Boot Assistant protocol; what bossac speaks.
- **SWD** -- Serial Wire Debug. ARM's two-wire debug interface, used on Cortex-M.

1.20.5 Networking / WiFi-specific

- **ATWINC1500** -- Microchip's WiFi controller IC, on the MKR1000.
- **ATSAMW25** -- the module containing SAMD21 + ATWINC1500.
- **BSP** -- Board Support Package. The platform-specific glue between a generic driver and your hardware.
- **HAL** -- Hardware Abstraction Layer.
- **lwIP** -- lightweight TCP/IP stack. Used in embedded systems if you want your own stack instead of the WINC1500's on-chip one.
- **SPI** -- Serial Peripheral Interface. The bus the SAMD21 uses to talk to the WINC1500.
- **UART** -- Universal Asynchronous Receiver/Transmitter. Classic 2-wire serial port (plus ground).

1.20.6 Other ISAs / vendors

- **AVR** -- 8-bit MCU family originally from Atmel, now Microchip.
- **MIPS / PowerPC / SuperH** -- legacy 32-bit RISC architectures.
- **MSP430** -- TI's 16-bit MCU family.
- **PIC** -- Microchip's older 8/16/32-bit MCU families.
- **RISC-V** -- open instruction set architecture.
- **Xtensa** -- Tensilica's configurable ISA, used in ESP8266/ESP32.
- **x86 / x86-64** -- Intel/AMD desktop CPU architecture.

1.21 Appendix B -- GCC flags reference for bare-metal ARM

A cheat-sheet of the flags you'll meet (and want to meet) when compiling and linking for the SAMD21. Most apply unchanged to any Cortex-M target. Organised by purpose.

1.21.1 Target architecture flags (pick a CPU/ISA)

Flag	What it does	Notes
`-mcpu=cortex-m0plus`	Selects core: tunes scheduling, allowed instructions, calling convention	Use the exact core name. Wrong choice = wrong instructions or suboptimal code.
`-mthumb`	Generate Thumb instructions	Mandatory for Cortex-M (it only runs Thumb). With `-mcpu=cortex-m*` it's implicit but be explicit.
`-mfloat-abi=soft`	Software floating point, no FPU instructions	M0+ has no FPU, so `soft` is the only valid choice. M4F/M7F can use `softfp` or `hard`.
`-mfpu=none`	No FPU	Default on M0+; spell it out for clarity.
`-mlittle-endian`	Little-endian (ARM default)	Rarely changed.

Get these wrong and your code may compile but fault at runtime with illegal instructions.

1.21.2 Language standard and dialect

Flag	What it does
`-std=c11` / `-std=c17` / `-std=c23`	Select C standard. `c11` is a safe default; `gnu11`/`gnu17` enables GNU extensions.
`-std=gnu++17`	C++ with GNU extensions (Arduino uses `gnu++11`).
`-ffreestanding`	Tells the compiler this is not a hosted environment: no assumption that `main` returns to the OS, no built-in `printf` optimisations, etc. **Always use on bare metal.**
`-fno-builtin`	Don't replace calls like `strlen` with compiler builtins. Useful when you implement them yourself.

Flag	What it does
`-fno-common`	Treat tentative definitions strictly (no implicit COMMON section). Catches duplicate globals at link time. Default in newer GCC.
`-fshort-enums`	Enums use the smallest type that fits. Saves RAM but breaks ABI with code compiled without it. **All your code must agree.**

1.21.3 Optimisation levels

Flag	What it does	When to use
`-O0`	No optimisation, fastest compile, easiest to debug	While actively debugging with GDB
`-O1`	Light optimisation	Rarely chosen explicitly
`-O2`	Standard release optimisation	Default for production code
`-O3`	Aggressive (inlining, vectorisation hints)	Often *worse* on tiny MCUs because code grows past flash; benchmark
`-Os`	Optimise for **size**	The right default for Cortex-M0+ with 256 KiB flash. Arduino uses `-Os`.
`-Og`	Optimise but keep debuggability	Best of both for "release build I want to debug"
`-Oz` (Clang)	Even more size than `-Os`	LLVM only.

Practical: develop with `-Og -g3` (debuggable but optimised), produce release with `-Os -g3` (keep symbols, strip later with `objcopy --strip-debug`).

1.21.4 Size-saving flags (worth the cognitive cost on MCUs)

Flag	What it does
`-ffunction-sections`	Put every function in its own ELF section
`-fdata-sections`	Put every global in its own ELF section
`-Wl,--gc-sections`	Linker: discard sections nothing references

These three together typically shave 20-50% off bare-metal binaries. They are what makes `KEEP(...)` in your linker script necessary for the vector table.

`-fno-rtti` (C++) | Disable run-time type info | Saves several KiB |
`-fno-exceptions` (C++) | Disable C++ exceptions | Saves a lot; mandatory on embedded C++ usually |
`-fno-threadsafe-statics` (C++) | Don't emit guards around static initialisers | OK in single-threaded firmware |
`--specs=nano.specs` | Use newlib-nano | Major libc size reduction |
`--specs=nosys.specs` | Use empty syscall stubs | Lets newlib link without an OS |
`-fno-unwind-tables` | No stack unwinding tables (for exceptions/backtraces) | Saves space if you don't need backtrace |
`-fno-asynchronous-unwind-tables` | Same for async unwind tables | Saves space |
`-fmerge-all-constants` | Merge identical literals across compilation units | Small wins |

1.21.5 Debug information

Flag	What it does
`-g`	Default debug info (DWARF)
`-g3`	Maximum debug info, including macro definitions
`-gdwarf-4` / `-gdwarf-5`	Pick DWARF version (5 is current; some old GDBs only handle 4)
`-ggdb`	Tune debug info for GDB specifically

Debug info lives in the ELF, never in your `.bin`/`.hex`. There's no flash-size cost to compiling with `-g3`. **Always compile with debug info on**, strip for release if you must, but keep the ELF.

1.21.6 Warnings -- the ones you actually want on

GCC's default warnings are weak. Turn more on:

Flag	What it does
`-Wall`	A starter set. Misnamed -- it's not "all warnings."
`-Wextra`	A bigger starter set than `-Wall`. Use both.
`-Wpedantic`	Strictly conform to the chosen `-std=`. Catches GNU extension drift.
`-Wshadow`	Variable shadowing (e.g. inner `int i` hiding outer).
`-Wconversion`	Implicit narrowing conversions (`int -> uint8_t` and friends). Noisy but valuable on MCUs.
`-Wsign-conversion`	Implicit signed-unsigned conversions.
`-Wdouble-promotion`	Catches accidental `double` arithmetic on M0+ (which has no FPU at all, so even `float` is software).
`-Wfloat-equal`	`==` on floats; usually a bug.
`-Wstrict-prototypes`	C: refuse `f()` (unspecified args), require `f(void)` for no-args.
`-Wmissing-prototypes`	Catches functions defined without a preceding declaration -- usually missing `static`.
`-Wundef`	`#if FOO` when `FOO` is undefined evaluates to 0 silently; this warns instead.
`-Wcast-align`	Casts that increase required alignment (`uint8_t*` -> `uint32_t*`).
`-Wcast-qual`	Casts that drop `const`/`volatile`.
`-Wnull-dereference`	Static null-pointer use.
`-Wformat=2`	Stricter `printf`/`scanf` format checks.
`-Wlogical-op`	Suspicious `&&`/`||` (e.g. `x && x`).
`-Wredundant-decls`	Same declaration repeated.
`-Wstack-usage=N`	Warn if any function uses more than N bytes of stack. Set N to a fraction of your stack region.

For very strict code: add `-Werror` to *promote warnings to errors*. Recommended once a project stabilises -- it forces you to deal with warnings instead of ignoring them.

Specific to bare metal and `volatile` registers, also useful:

Flag	What it does
`-Wvolatile-register-var`	Catches `volatile` on a register-qualified variable (a bug).
`-Wpadded`	Warns when the compiler adds struct padding. Critical when overlaying structs on hardware register layouts -- you want zero surprise padding.

1.21.7 Static analysis (free, on by default in newer GCC)

Flag	What it does
`-fanalyzer`	GCC's built-in static analyser (since GCC 10). Finds use-after-free, leaks, double-free, taint issues. Slow; run in CI not every build.
`-fsanitize=undefined`	Compile-time + runtime undefined-behaviour checks. Useful in unit tests on the host; needs runtime support to link on bare metal.

1.21.8 Linker-specific flags (pass via `-Wl,<flag>`)

Flag	What it does
`-T linker.ld`	Use this linker script
`-Wl,--gc-sections`	Discard unused sections
`-Wl,-Map=output.map`	Emit a linker map file. **Always emit this; it's how you find size hogs.**
`-Wl,--cref`	Add cross-reference table to map file
`-Wl,--print-memory-usage`	Print FLASH/RAM utilisation summary after link
`-Wl,--check-sections`	Verify sections don't overlap
`-Wl,--unresolved-symbols=report-all`	Be loud about unresolved symbols
`-Wl,--warn-common`	Warn on COMMON symbols (paired with `-fno-common` in compile)
`-Wl,--warn-section-align`	Warn if a section's alignment forces an address shift
`-Wl,--build-id=none`	Don't embed a build-ID note (slightly smaller flash)
`-nostdlib`	Don't link standard libraries
`-nostartfiles`	Don't link `crt0` -- you provide your own (Module 13).
`-nodefaultlibs`	Don't link default libraries (libgcc, libc)

1.21.9 A reasonable default flag-set for this project

A `Makefile` skeleton you can grow into:

```
CPU      = cortex-m0plus
CFLAGS   = -mcpu=$(CPU) -mthumb -mfloat-abi=soft \
           -std=c11 -ffreestanding -fno-common \
           -ffunction-sections -fdata-sections \
           -Os -g3 -gdwarf-5 \
           -Wall -Wextra -Wpedantic \
```

```
            -Wshadow -Wconversion -Wdouble-promotion \
            -Wundef -Wcast-align -Wcast-qual \
            -Wmissing-prototypes -Wstrict-prototypes \
            -Wformat=2 -Wnull-dereference -Wpadded \
            -Wstack-usage=512

LDFLAGS  = -mcpu=$(CPU) -mthumb -nostdlib -nostartfiles \
            -T linker.ld \
            -Wl,--gc-sections \
            -Wl,-Map=$(BUILD)/firmware.map \
            -Wl,--cref \
            -Wl,--print-memory-usage \
            -Wl,--check-sections \
            -Wl,--warn-common \
            -Wl,--warn-section-align
```

Action: at the end of every build, **read the map file**. Look at the `Memory Configuration` section to see flash/RAM headroom. Look at section sizes to find which translation unit is the biggest. This habit alone separates serious embedded developers from beginners.

1.21.10 Debugging the build itself

When something goes wrong at compile/link time:

Flag	What it does
`-v`	Print every subprocess GCC runs. Tells you which `as`, `ld`, `crt0`, specs file got picked.
`--save-temps`	Keep `.i` (preprocessed), `.s` (assembly), `.o` after compilation
`-E`	Stop after preprocessing (output expanded source to stdout)
`-S`	Stop after compilation (output `.s` assembly)
`-M` / `-MMD`	Generate make-style dependency files
`-print-multi-lib`	Show which multilibs GCC has for the current target (relevant when picking `-mcpu`)
`-print-search-dirs`	Where GCC is looking for libraries and includes
`-print-file-name=libc.a`	Resolve a library name to its full path
`-Q --help=target`	List all target-specific flags GCC understands for your config

Useful trick: `arm-none-eabi-gcc <your flags> -E -dM -x c /dev/null` prints every predefined macro for your target. Lets you check that `__ARM_ARCH_6M__`, `__SAMD21G18A__`, etc. are defined as expected.

1.21.11 Post-link inspection (binutils companions)

These aren't compile flags but you'll use them every build:

Tool	What it does
`arm-none-eabi-size -A firmware.elf`	Per-section byte counts
`arm-none-eabi-size -G firmware.elf`	GNU summary format
`arm-none-eabi-objdump -d firmware.elf`	Disassemble everything
`arm-none-eabi-objdump -h firmware.elf`	Section headers
`arm-none-eabi-objdump -t firmware.elf`	Symbol table
`arm-none-eabi-objdump -S firmware.elf`	Source-interleaved disassembly (needs `-g`)
`arm-none-eabi-nm --size-sort firmware.elf`	Symbols sorted by size -- find the fat ones
`arm-none-eabi-readelf -a firmware.elf`	Verbose ELF dump
`arm-none-eabi-objcopy -O binary firmware.elf firmware.bin`	Strip to raw binary
`arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex`	Convert to Intel HEX
`arm-none-eabi-objcopy --strip-debug firmware.elf firmware-release.elf`	Strip debug info for distribution

1.22 Appendix C -- UART, I2C, SPI: bus protocols with practical examples

These three buses cover ~95% of the on-board and off-board peripherals you'll ever talk to. The SAMD21 implements all three with the same peripheral block: **SERCOM** (Serial Communication), configured into the mode you need. The protocol-level concepts in this appendix are **bus-agnostic**: they apply to any UART/I2C/SPI peripheral on any MCU. The MKR1000-specific details (which pins, which SERCOM) are pointers into your datasheets.

The practical examples target the three on-board peripherals from Module 0: - a **debug UART** to a USB-serial dongle (you've done this already in Module 11); - **I2C to the ATECC508A** crypto chip; - **SPI to the ATWINC1500** WiFi controller (overlaps with Module 14).

1.22.1 C.1 UART (Universal Asynchronous Receiver/Transmitter)

1.22.1.1 Concept A UART is a **point-to-point, asynchronous, full-duplex** serial link. Two devices each have a TX (transmit) and RX (receive) line; you cross them: **A.TX -> B.RX** and **A.RX -> B.TX**. Plus a common ground.

"Asynchronous" means there is **no shared clock wire** -- both sides have to be configured with the same **baud rate** (bits per second) and they re-sync per byte on the falling edge of the start bit.

1.22.1.2 The wire-level frame One byte sent on a UART line looks like this on the wire:

```
idle (high) | START | bit0 | bit1 | bit2 | bit3 | bit4 | bit5 | bit6 | bit7 | [parity] | STOP | i
            |  low  |   ----- data, LSB first ------------------------------ |          | high
```

- **Idle** state: line held high.
- **Start bit**: line goes low for one bit-time -- the receiver's edge trigger.
- **Data bits**: usually 8, sometimes 7 or 9. LSB transmitted first.
- **Parity** (optional): even/odd/none. Almost everyone uses **none** ("8N1" = 8 data, no parity, 1 stop).
- **Stop bit**: line returns high for at least one bit-time.

If both sides agree on the same baud rate to within ~3%, the bytes arrive intact. Mismatched baud is the #1 newcomer problem and looks like garbage characters on screen.

1.22.1.3 Speeds and signalling levels

- Common baud rates: 9600, 19200, 38400, 57600, **115200**, 230400, 460800, 921600. Higher than ~1 Mbps becomes flakey on long wires.
- **TTL UART** (what the SAMD21 generates): 0--3.3 V (or 0--5 V on AVR). This is what your MKR1000 puts on its TX pin.
- **RS-232** (a *different* electrical layer carrying UART frames): +/-3 to +/-15 V, inverted. Used by old PCs. Need a level shifter (MAX3232 etc.) to connect TTL to RS-232.
- **RS-485**: differential pair, multi-drop, up to ~1 km. Same UART framing on top.

You will use TTL-level UART here. If you connect to a PC, you go through a **USB-to-UART bridge** (CP2102, FT232R, CH340) which presents as `/dev/ttyUSB0` on Linux.

1.22.1.4 Use cases

- **Debug `printf`** -- the canonical use. Slow, simple, visible.
- **MCU-to-MCU short links** between two boards you control.
- **GPS modules, GSM modems, some sensors** -- typically NMEA or AT-command protocols at 9600--115200 baud.
- **Bluetooth modules** (HC-05/HC-06) -- UART on the MCU side, Bluetooth on the air side.
- **Console of a Linux SBC** -- the Pi's debug UART, see the Linux course Module 5.

1.22.1.5 Practical example: debug printf on the MKR1000 Already covered in Module 11. Recap with concrete steps:

1. Pick one SERCOM (Arduino normally uses **SERCOM5** for the board's broken-out TX/RX pins -- confirm in `docs/ABX00004/ABX00004-full-pinout.pdf`).
2. Configure that SERCOM in **USART** mode. Key registers (look up exact names in the SAMD21 SERCOM-USART chapter):
   - CTRLA: select async, set TX/RX pad numbers, internal clock, LSB first.
   - CTRLB: 8-bit, 1 stop bit.
   - BAUD: the fractional baud formula in the datasheet. For 115200 baud from a 48 MHz GCLK, compute and write the right 16-bit value.
3. Set the GPIO pins for those pads to **peripheral function** so the SERCOM controls them instead of PORT.
4. Enable the peripheral, then enable TX (and RX if you want input).
5. To send a byte: poll the `DRE` (Data Register Empty) flag, then write the byte to the DATA register.

Connect:

```
MKR1000 TX  ---- USB-UART RX
MKR1000 GND ---- USB-UART GND
[optional]  ---- USB-UART TX --- MKR1000 RX
```

On Linux:

```
picocom -b 115200 /dev/ttyUSB0
```

Common bug: forgetting that the MKR1000's TX/RX pin labels refer to **the MCU's** TX and RX. Many USB-UART adapters label theirs the same way. **TX on one side goes to RX on the other.**

Action: run your interrupt-driven UART driver (from RTOS course Module 10, or the simpler polled version from bare-metal Module 11) and confirm a one-second heartbeat prints cleanly.

1.22.2 C.2 I2C (Inter-Integrated Circuit)

1.22.2.1 Concept I2C is a **two-wire, synchronous, multi-master, multi-slave, addressed** bus. Both wires are **open-drain** with pull-up resistors -- any device can pull a wire low, no device drives it high. This is what enables multi-device sharing on the same pair of wires.

The two wires: - **SDA** (Serial Data) -- bidirectional data line. - **SCL** (Serial Clock) -- clock line, normally driven by the master.

Plus a common ground.

1.22.2.2 Pull-up resistors Critical to understand: SDA and SCL are not driven high. They are held high by **external pull-up resistors** (typically 1.5 k Ohm to 10 k Ohm) to Vcc. A typical hobbyist value is 4.7 k Ohm.

- Too weak (high resistance): the line takes a long time to rise back to high after a low pulse; you see rounded edges on a scope; data errors at higher speeds.
- Too strong (low resistance): excessive current draw, possibly out of spec for the driving open-drain transistor.

The Pi 4 has **internal 1.8 k Ohm pull-ups** on its I2C pins; the SAMD21's internal pull-ups are much weaker (10s of k Ohm) and usually not adequate. **Most boards that expose I2C externally include explicit pull-up resistors.** Check whether your sensor breakout board already has them before adding more.

1.22.2.3 The wire-level transaction

```
START | ADDR(7) | R/W(1) | ACK | DATA(8) | ACK | DATA(8) | ACK | ... | STOP
```

- **START condition**: SDA falls while SCL is high.
- **7-bit address**: most common; identifies the slave. (10-bit addresses also exist; rare.)
- **R/W bit**: 0 = master writes, 1 = master reads.
- **ACK bit**: the *receiver* (slave for writes, master for reads) pulls SDA low for one clock to acknowledge.
- **Data bytes**: 8 bits each, MSB first, each followed by ACK from the receiver.
- **STOP condition**: SDA rises while SCL is high.

Multi-byte reads typically work as:

```
START | ADDR | W | ACK | REG | ACK | RESTART | ADDR | R | ACK |
       <-- "write register pointer" --->     <-- read from there -->
   DATA0 | ACK | DATA1 | ACK | ... | DATAn | NAK | STOP
```

The master signals "last byte" with a **NAK** (not pulling SDA low on the ACK slot).

1.22.2.4 Speeds

- **Standard mode**: 100 kHz. Bulletproof, works with 4.7 k Ohm pull-ups, long-ish wires.
- **Fast mode**: 400 kHz. Most modern sensors support this.
- **Fast mode plus**: 1 MHz. Needs stronger pull-ups, shorter wires.
- **High-speed mode**: 3.4 MHz. Rare; requires specific signalling.

Start with 100 kHz to debug, move to 400 kHz once it works.

1.22.2.5 Use cases

- **Sensors**: temperature (BME280, LM75), accelerometers (MPU6050, LIS3DH), magnetometers, light sensors -- the I2C catalogue is enormous.
- **EEPROMs**: 24Cxx family.
- **RTCs**: DS1307, DS3231, PCF8523.
- **Small OLED displays**: SSD1306 0.96" OLEDs.
- **Crypto chips**: the **ATECC508A on the MKR1000**.
- **Port expanders**: MCP23017 (16 GPIO over I2C).

Multi-drop short distance. Beyond a few tens of cm of wire, switch to something differential.

1.22.2.6 Practical example: read the ATECC508A's serial number The MKR1000's crypto chip is at a fixed I2C address on the SAMD21's internal I2C bus (the bus is *not* the one routed to the external header). Confirm:

- The I2C address of the ATECC508A (Microchip datasheet -- default is typically `0x60` for ATECC508A; verify on your board).
- Which SAMD21 SERCOM is wired to it (schematic).
- Which pins of that SERCOM are SDA/SCL (datasheet pinout for the chosen SERCOM).

Steps to get a "hello" from the chip:

1. Configure the right SERCOM in **I2C master** mode. Key registers:
   - CTRLA: I2C master, set speed, define inactive timeout.
   - BAUD: from the datasheet's I2C baud formula.
   - CTRLB: smart mode = 1 (auto-ACK by hardware) makes the API less painful.
2. Set the SDA/SCL GPIO pins to peripheral function.
3. Enable, wait for BUS state to enter IDLE.
4. Send the **wake** sequence: ATECC508A is normally sleeping; you wake it by holding SDA low for >60 us (datasheet says "tWLO"), which the chip's wake-detect circuitry sees. After this, the chip responds to its I2C address.
5. Address it: write `0x60 << 1 | 0` (write) and a "Info" command packet per the ATECC508A datasheet's command format (each command is: count, opcode, params, CRC).
6. The chip sets a busy flag while computing; poll for completion.
7. Read back the response: write address + read bit, read N bytes, NAK the last.

This is a non-trivial first project. **Strongly recommended** to get I2C working against a simpler device first -- e.g. an MCP23017 port expander or a BMP280 -- both of which are "write register, read register" with no wake/CRC dance. Then come back to the ATECC508A.

1.22.2.7 Tools that will save your sanity

- A **logic analyser** (Saleae clone, ~20 EUR) with I2C decoding. Looking at SCL and SDA on a scope is almost useless; looking at the decoded transaction is instant clarity.
- On Linux, `i2cdetect`, `i2cget`, `i2cset`, `i2cdump` (from the `i2c-tools` package) -- run on a Pi connected to the same bus as your MCU's slave, you can probe devices without writing firmware.

  Action: before touching the ATECC508A, talk to *any* I2C sensor you have lying around from the MKR1000. Read its `WHO_AM_I` register. When the right value comes back, your I2C is real.

1.22.3 C.3 SPI (Serial Peripheral Interface)

1.22.3.1 Concept SPI is a **four-wire, synchronous, full-duplex, single-master, multi-slave** bus. Unlike I2C, there is **no addressing** -- each slave has its own dedicated **chip-select (CS)** wire that the master pulls low to talk to that specific slave.

The wires: - **SCK** (or SCLK) -- serial clock, driven by the master. - **MOSI** (Master Out, Slave In) -- master sends data to slave. - **MISO** (Master In, Slave Out) -- slave sends data to master. - **CS** (or **SS**, or **NSS**) -- chip select, active-low, one per slave.

Plus ground. No pull-ups needed; everything is push-pull. No address phase; just shift bits.

1.22.3.2 Full duplex shifting The master clocks bits out on MOSI and **simultaneously** clocks bits in on MISO. Conceptually there is one giant shift register made of the master's TX and the slave's TX joined in a ring. Every SCK edge: master shifts a bit out on MOSI, slave shifts a bit out on MISO.

You always send and receive the same number of bytes. If you only want to read, you write dummy bytes (often `0xFF` or `0x00`).

1.22.3.3 Modes: CPOL and CPHA The four "SPI modes" describe when the bit is valid relative to the clock edge:

Mode	CPOL	CPHA	Idle clock	Bit valid on
0	0	0	low	leading (rising) edge
1	0	1	low	trailing (falling) edge
2	1	0	high	leading (falling) edge
3	1	1	high	trailing (rising) edge

Each slave's datasheet specifies which mode. Mode 0 is the most common. The WINC1500 uses **Mode 0**. If the mode is wrong, you'll see shifted-by-half-a-bit nonsense.

1.22.3.4 Speeds SPI is **much faster** than UART or I2C. Common ranges:

- **1--10 MHz**: most sensors and displays.
- **10--40 MHz**: SD cards, fast flash, the WINC1500.
- **40 MHz+**: high-end ADCs, dedicated chips. Wiring quality starts to matter.

Speed is set by a divider in the master's clock-rate register.

1.22.3.5 CS handling Single-slave: tie CS low permanently if the slave allows it (some do, many don't); otherwise drive it from a GPIO.

Multi-slave: each slave has its own GPIO CS, and the master selects one at a time. Daisy-chained slaves exist but are uncommon.

Common bug: forgetting to keep CS low for the whole multi-byte transaction. If your driver toggles CS per byte, the slave thinks each byte is a new command. Wrap the entire transfer in "assert CS / shift bytes / de-assert CS".

1.22.3.6 Use cases

- **WiFi modules** -- including the WINC1500 on your MKR1000.
- **SD cards** (in SPI mode -- SD cards also have a native 4-bit mode).

- **External flash** (W25Qxx and friends).
- **High-speed sensors** (ADXL345 in SPI mode, fast ADCs).
- **TFT and OLED displays** (ST7735, ILI9341, SSD1351).
- **CAN controllers** (MCP2515).

Short-distance only. Past a few tens of cm, signal integrity dies.

1.22.3.7 Practical example: read the WINC1500's chip ID Concrete and useful: first SPI message you'll send to the WINC1500. Detailed in bare-metal Module 14.3 step 2; here's the SPI side of it stripped of WINC specifics.

1. Configure SERCOM in **SPI master** mode:
   - CTRLA: SPI master, MOSI/MISO pad numbers (depends on which SERCOM and which pads are wired to WINC -- check schematic), CPOL=0, CPHA=0 (mode 0), MSB first.
   - CTRLB: receive enabled, slave select pin manual (drive CS via GPIO yourself for predictable framing).
   - BAUD: SAMD21 SPI baud register; pick a conservative 1--4 MHz for first contact, raise later.
2. Set SCK, MOSI, MISO pins to peripheral function. Set CS as a plain GPIO output.
3. Sequence to read a 32-bit register from the WINC1500:

```
gpio_clear(WINC_CS);
spi_transfer_byte(CMD_REG_READ);   // WINC-specific opcode
spi_transfer_byte((addr >> 16) & 0xFF);
spi_transfer_byte((addr >>  8) & 0xFF);
spi_transfer_byte((addr >>  0) & 0xFF);
// optional CRC/dummy bytes per WINC protocol
uint8_t b0 = spi_transfer_byte(0xFF);  // dummy to read
uint8_t b1 = spi_transfer_byte(0xFF);
uint8_t b2 = spi_transfer_byte(0xFF);
uint8_t b3 = spi_transfer_byte(0xFF);
gpio_set(WINC_CS);
```

4. Print the assembled 32-bit result. Compare to the datasheet's expected chip-ID value.

For a **before-WINC** sanity check, do **SPI loopback**: jumper MOSI directly to MISO with a wire, and verify `spi_transfer_byte(x) == x` for several `x`. This confirms your SPI peripheral is alive independently of any external slave.

1.22.3.8 Tools

- Logic analyser with SPI decoding is again indispensable.
- For loopback testing: literally a piece of bare wire from MOSI to MISO.

  Action: get SPI loopback working before doing anything WINC-related. If loopback fails, no WINC code will ever work.

1.22.4 C.4 Side-by-side comparison

Property	UART	I2C	SPI
Wires (plus GND)	2 (TX, RX)	2 (SDA, SCL)	3 + NxCS
Synchronous?	No (async, baud)	Yes (SCL)	Yes (SCK)
Duplex	Full	Half	Full
Number of slaves	1 (point-to-point)	Many (7-bit address)	Many (one CS each)
Typical speed	9.6--921.6 kbps	100 k / 400 k / 1 M Hz	1--40 MHz
Pull-ups needed	No	**Yes** (external)	No
Hardware addressing	None	Yes (built into frame)	None (use CS)
Wire length	up to ~10 m at low baud (TTL); km with RS-485	tens of cm	tens of cm
ACK on the wire	No	Yes	No
Clock arbitration	n/a	Yes (multi-master capable)	n/a
MKR1000 example	debug UART	ATECC508A crypto	ATWINC1500 WiFi

1.22.5 C.5 How to pick which to use

When designing a new peripheral connection:

- **One link, simple data, low speed, ASCII or trivially-framed binary?** -> UART. Easy to debug; you can put a USB-UART adapter on it and `picocom` to peek at traffic.
- **Multiple low-rate sensors, want shared wires?** -> I2C. The addressing means multi-drop just works.
- **High data rate, short distance, no addressing concerns?** -> SPI. The fastest of the three by a lot.
- **You need to *bridge* two boards on different power domains?** -> UART with explicit GND tie, optionally optoisolated. Or CAN. I2C/SPI across power domains is fiddly.

1.22.6 C.6 What to debug, in what order

Whatever the bus, when "it doesn't work":

1. **GND**. Both sides must share ground. Check with a multimeter.
2. **Voltage levels**. 3.3 V vs 5 V mismatches blow ports or fail to clock data.
3. **Pin numbers**. The most embarrassing class of bug. Confirm against the SAMD21 datasheet's *pad-to-pin* table for the SERCOM you picked and against the schematic for what's actually wired.
4. **Peripheral function set on GPIO**. The pin is by default a PORT GPIO; until you select the peripheral mux, SERCOM cannot reach it.
5. **Clock to the peripheral**. GCLK + PM must both be configured.
6. **Bus configuration**. UART: baud. I2C: speed, pull-ups. SPI: mode (CPOL/CPHA), speed, bit order.
7. **For I2C: pull-ups present?** Measure SDA and SCL idle voltage -- should be Vcc. If they sit at 0 V or float, you're missing pull-ups.
8. **For SPI: loopback works?** MOSI-MISO jumper, transfer byte, compare in == out. Eliminates the slave.

A logic analyser shortcuts all of the above. Stop trying to debug serial protocols blind.

1.23 Appendix D -- Designing your own PCB: open-source tools and a git workflow

If you eventually want a custom MKR1000 carrier board, a breakout, or your own MCU board from scratch, you'll want a PCB design tool. This appendix covers Linux-friendly, locally installable, open-source options that play well with git. **Nothing here is a web app**; everything runs on your machine and stores its project as files you control.

1.23.1 The graphical EDA tools

Tool	Maturity	License	File format	Strengths	Weaknesses
KiCad	Production-grade; the de-facto open-source EDA	GPLv3	S-expressions (text) since KiCad 6+	Huge community, vast part libraries, well-documented, native Linux, supports import from Altium/Eagle	GUI-heavy; large project files
Horizon EDA	Mature, Linux-first	GPLv3	JSON / text	Genuinely "designed for power users on Linux" -- keyboard-driven, monotone UI, single-database parts model, very git-friendly	Smaller part library; smaller community
LibrePCB	Stabilising	GPLv3	XML / text	Clean modern UI, library-versioning baked in, cross-platform	Less feature-complete than KiCad for advanced routing
pcb-rnd	Niche	GPL	text (lihata)	Continuation of gEDA PCB; very scriptable; old-school	Schematic capture is separate (gschem or sch-rnd); steeper learning curve
gEDA / gschem + PCB	Legacy	GPL	text	Pure-CLI possible; classic Unix philosophy	Largely superseded by KiCad/pcb-rnd; minimal active development

Recommendation for you, today: KiCad. It is the safest default, has the largest body of online tutorials, and since KiCad 6 its files are S-expression text -- meaningfully diffable in git. KiCad 8 is the current stable line; KiCad 9 is recent. If you find KiCad too mouse-heavy after a few weeks, try Horizon EDA as a "Linux power-user's EDA."

1.23.2 Code-as-hardware tools (for CLI lovers)

If you'd rather *describe* circuits in code than draw them, these are worth knowing. They typically generate netlists or KiCad files as output, so they slot into a normal PCB flow:

- **atopile** -- domain-specific language (`.ato` files) for describing circuits as modules. Generates KiCad-compatible netlists.
  - *Pro*: package-manager mindset, designed for git, strong CLI; the most "hardware as code" feel of the bunch.
  - *Con*: young; smaller community; standard libraries still growing.
  - *Pick when*: you want hardware-as-code and you're starting fresh.
- **SKiDL** -- Python library for describing schematics. Generates KiCad netlists.
  - *Pro*: mature; if you know Python you're productive immediately; plays well with KiCad libraries you already have.
  - *Con*: Python syntax for hardware is less natural than a DSL; less structured than atopile.
  - *Pick when*: you already think in Python and want a thin code layer over KiCad.
- **tscircuit** -- TypeScript/JSX (React-like) for circuit description. Toolchain runs locally despite the web-stack flavour.
  - *Pro*: familiar to web developers; component-tree model is expressive.
  - *Con*: newest of the four; smallest hardware-engineering community.
  - *Pick when*: you're a web developer who occasionally does hardware.
- **PCBmodE** -- Python + JSON to SVG, then to Gerbers.
  - *Pro*: unique workflow for artistic / unusual board outlines and silkscreen.
  - *Con*: non-standard output path (no schematic editor); niche.
  - *Pick when*: you're making Boldport-style art boards.

These pair well with a graphical tool: describe the schematic in atopile or SKiDL, then open the generated netlist in KiCad's PCB editor to place and route. You get text-first reproducibility plus the GUI for the spatial part that doesn't want to be code.

1.23.3 Utilities every PCB-on-Linux workflow benefits from

- **gerbv** -- open-source Gerber viewer. Confirm your fab output before sending to a board house.
- **FlatCAM** -- generate CNC G-code from Gerbers, if you ever mill PCBs at home.
- **OpenSCAD + KiCad's STEP/VRML export** -- mechanical sanity checks (does the PCB fit the enclosure?). FreeCAD's KiCad StepUp workbench is the bridge.
- **ngspice** -- SPICE simulator. KiCad integrates with it for schematic- level simulation.
- **kicad-cli** -- KiCad 7+ ships a real CLI (`kicad-cli`) for headless ERC, DRC, plotting, and 3D export. This is what makes CI pipelines possible.

1.23.4 Putting PCB design under git

KiCad's text format is git-friendly, but a project still has gotchas. A practical setup:

1.23.4.1 Project layout

```
my-board/
```

```
|-- my-board.kicad_pro          # project file (text)
|-- my-board.kicad_sch          # schematic (text)
|-- my-board.kicad_pcb          # board layout (text)
|-- my-board.kicad_prl          # local UI state -- DO NOT COMMIT
|-- sym-lib-table               # per-project symbol libraries
|-- fp-lib-table                # per-project footprint libraries
|-- lib/                        # vendored custom symbols/footprints
|   |-- my-board.kicad_sym
|   |-- my-board.pretty/
|-- 3dmodels/                   # vendored STEP files (consider git-lfs)
|-- output/                     # gerbers, BOM, drill -- generated, DO NOT COMMIT
\-- docs/                       # schematics PDF, photos, notes
```

1.23.4.2 `.gitignore`

```
# KiCad
*.kicad_pro-bak
*.kicad_sch-bak
*.kicad_pcb-bak
*-backups/
fp-info-cache
*.kicad_prl
*.lck
_autosave-*

# Generated artefacts
output/
*.gbr
*.drl
*.pos
*.csv

# OS / editor
.DS_Store
*~
.*.swp
```

The `_prl` (project local) file holds UI state like window positions -- it churns on every edit and should not be committed.

1.23.4.3 Vendor your libraries The number-one cause of "the board changed when I cloned it on another machine" is using KiCad's global libraries. Solution: **make every symbol and footprint you use part of the repo**, under `lib/`. Set the project's `sym-lib-table` and `fp-lib-table` to use only project-relative paths. This makes your repo reproducible and immune to KiCad library reorganisations.

1.23.4.4 `git-lfs` for binary attachments 3D STEP files, datasheet PDFs in `docs/`, and reference photos compress poorly and bloat the repo. Use `git-lfs` for `*.step`, `*.stp`, `*.pdf`, `*.png` if/when the repo grows.

1.23.4.5 Visual diffs Text diffs of `.kicad_pcb` are technically possible but not human-friendly. Two tools help:

- **KiCad's "Compare with Previous Version"** built-in (KiCad 8+) renders a visual diff inside the editor.
- **kidiff** (third-party CLI tool) generates side-by-side PNG renders of any two git commits. Great for PR-style review even if you work alone.

1.23.4.6 Pre-commit hook: ERC/DRC must pass Use `kicad-cli` to enforce that every commit passes Electrical Rule Check and Design Rule Check:

```
#!/usr/bin/env bash
# .git/hooks/pre-commit (or use the `pre-commit` framework)
set -e
kicad-cli sch erc --exit-code-violations my-board.kicad_sch
kicad-cli pcb drc --exit-code-violations my-board.kicad_pcb
```

This catches "I forgot to wire up a net" before it's in history.

1.23.4.7 Versioning the manufactured artefacts When you order a board, tag the commit and store the Gerber zip elsewhere (not in the repo). E.g.:

```
git tag -a rev-a -m "Sent to fab 2026-05-24, OSHPark order #12345"
kicad-cli pcb export gerbers -o output/rev-a/ my-board.kicad_pcb
zip -r my-board-rev-a.zip output/rev-a/
# upload zip to your fab, archive it outside the repo
```

The tag is the source of truth for "this is what got built."

1.23.5 Learning resources (free, no web-EDA tutorials)

These are the resources Linux CLI types tend to recommend. Prefer ones that teach concepts (signal integrity, layer stackups, BGA fanout) over ones that are pure click-through KiCad demos -- though you'll want some of each.

KiCad-specific tutorials

- **Official KiCad documentation** -- https://docs.kicad.org/ (Getting Started in KiCad is genuinely good).
- **"Getting To Blinky"** by Chris Gammell -- classic free video series taking a complete beginner through a KiCad project.
- **Phil's Lab** (YouTube) -- KiCad tutorials plus genuine engineering content (signal integrity, USB layout, RF). Highly recommended.
- **Digi-Key's "KiCad Like A Pro"** (YouTube + courses; the YouTube content is free).
- **Robert Feranec** (YouTube) -- professional high-speed PCB design; KiCad and Altium content.

PCB design fundamentals (tool-agnostic)

- **Eric Bogatin's signal-integrity lectures** -- search for his Teledyne LeCroy series on YouTube. The gold standard for *why* PCB layout matters.
- **Henry Ott's "Electromagnetic Compatibility Engineering"** -- the EMC bible. Library / used-book purchase.
- **"High-Speed Digital Design: A Handbook of Black Magic"** by Howard Johnson -- terse, dense, classic.
- **Rick Hartley's "How to Achieve Proper Grounding"** -- the one-hour lecture on YouTube every embedded designer should watch once.

- **PCB Stackup design** -- ICD Stackup Planner has free educational content; Multi-CB and JLCPCB publish their stackups openly, useful for matching layer counts to impedance targets.
- **Sparkfun's "PCB Basics" tutorial** -- gentle starter on terminology.
- **Adafruit's tutorials on KiCad** -- friendly walk-throughs.

Manufacturing / DFM

- **Bittele**, **JLCPCB**, **OSHPark**, **PCBWay** all publish their capability sheets (minimum trace/space, drill sizes, controlled- impedance options). Read at least one before designing for a target fab.
- **Limor "Ladyada" Fried's manufacturing series** on YouTube (Adafruit factory tours) -- intuition for what your design will go through.

Open-source reference designs to read

- **Olimex** publishes KiCad/Eagle sources for many of their boards.
- **Adafruit** publishes Eagle (and increasingly KiCad) sources for most breakouts.
- **Sparkfun** open-source designs ditto.
- **Arduino itself** -- the MKR1000 schematic in your repo is, in effect, a reference design you can study.

1.23.6 Recommended starter path for you

1. **Install KiCad 8 or 9 from Void's repos** (`xbps-query -Rs kicad`), plus `gerbv` for verification.
2. **Do "Getting To Blinky"** end to end -- gives you the muscle memory for the four KiCad subprograms.
3. **Reverse-engineer a small section of the MKR1000 schematic** into KiCad -- e.g. just the USB connector + the SAMD21's USB pins + decoupling caps. You'll learn how to make footprints and read pin tables.
4. **Design a tiny breakout** that does something you actually want -- a SWD breakout for the MKR1000 test points is a great first board, lets you do Module 12, and keeps the design small enough to finish.
5. **Set up the git workflow above** from day one. Tag the commit you send to fab.
6. **Order from a fab that accepts gerbers via web upload** (OSHPark, JLCPCB, Aisler, Eurocircuits) -- you avoid online EDAs but you don't avoid the manufacturer's web order page. That's a different thing.

After your first board comes back and works, then consider whether code- first tools like atopile fit your style. Don't start there: you need to have felt the pain of manual placement to understand what code-first is buying you.